Ecole Nationale Supérieure d'Informatique et d'Analyse des Systèmes - RABAT
Departement Web and Mobile Engineering

Filière : **Data and Software Sciences**

Rapport de Projet de fin d'année

Maalem : Application mobile intelligente de mise en relation Artisans & Particuliers au Maroc

---

Réalisé par :

AIT OUFKIR Khadija

BRAIDI Hajar

Jury :

Pr.CHEIKHI Laila(Encadrante)

Pr.ABNANE Ibtissam(Examinatrice)

Année universitaire 2025/2026

Remerciements

Tout d'abord, on remercie **ALLAH** de tout puissant de nous avoir donné la santé et la volonté d'entamer et de terminer ce projet.

Nous tenons à exprimer nos sincères remerciements à notre encadrant **Prof. CHEIKHI** pour son accompagnement, son écoute attentive, ses conseils judicieux et sa disponibilité tout au long de la réalisation de ce projet. Grâce à ses orientations précieuses, nous avons pu surmonter les difficultés et atteindre les objectifs fixés. Nous avons beaucoup appris de son expertise et de son expérience, et nous sommes reconnaissantes de l'opportunité qu'elle nous a offerte d'acquérir de nouvelles compétences.

Nous remercions également notre jurée **Prof. ABNANE** pour le temps qu'elle a consacré à l'évaluation de notre travail et pour ses commentaires constructifs et pertinents. Sa présence lors de notre soutenance nous a permis de présenter notre projet avec plus de confiance et d'assurance. Nous sommes très reconnaissantes pour l'ensemble des remarques et des critiques constructives qu'elle nous a faites, qui ont contribué à l'amélioration de notre travail.

Résumé

Ce projet vise à développer une application mobile de mise en relation entre des citoyens ayant besoin de services artisanaux et des artisans qualifiés au Maroc. L'application définit trois profils : le citoyen, qui publie des demandes de service ; l'artisan, qui envoie des offres de service et réalise les missions ; et l'administra teur, qui supervise la plateforme. Un modèle d'intelligence artificielle embarqué sur l'appareil permet la classification automatique de trois types de pannes do mestiques à partir de photos.
La conception repose sur une méthodologie incrémentale et itérative, soutenue par la modélisation UML. La mise en œuvre combine Kotlin/XML avec l'architecture MVVM, Firebase pour le backend temps réel, et React.js pour l'interface d'administration web. Cette approche garantit une solution fiable et évolutive, contribuant à la digitalisation du secteur artisanal marocain.

Mots clés : Développement Mobile, Mise en relation, UML, Firebase, Classification d'images

Abstract

This project aims to develop a mobile application that connects citizens in need of artisanal services with qualified craftsmen ("Maalems") in their geographic area in Morocco. The application defines three user profiles : the citizen, who publishes service requests ; the craftsman, who submits offers and carries out assignments ; and the administrator, who oversees the platform. An artificial intelligence model embedded on the device enables automatic classification of tree types of household issues from photos.

The design follows an incremental and iterative methodology, supported by UML modeling. The technical implementation combines Kotlin/XML with the MVVM architecture, Firebase for the real-time backend, and React.js for the web-based administration interface. This approach ensures a reliable and scalable solution, contributing to the digitalization of the Moroccan artisanal sector.

Keywords : Mobile Development, Matchmaking Platform, UML, Firebase, Image Classification

Liste des abréviations

Abréviation	**Désignation**
API	Application Programming Interface
CNN	Convolutional Neural Network
FCM	Firebase Cloud Messaging
MVVM	Model-View-ViewModel
SDK	Software Development Kit
TFLite	TensorFlow Lite
UML	Unified Modeling Language
XML	Extensible Markup Language

Table des figures

1.1 Cycle de vie de maalem 17
1.2 Diagramme de Gantt du projet Maalem 17
2.3 Diagramme des cas d'utilisations global 24
3.4 Diagramme de séquence : Postuler à une demande de service . . . 31
3.5 Le patron architectural MVVM 34
4.6 Distribution des classes 38
4.7 Jeu de données par classe 39
4.8 Variations generees par l'augmentation 41
4.9 Matrice de confusion 46
5.10 Interfaces de connexion et d'inscription 53
5.11 Écran d'accueil et création d'une demande (espace citoyen) 54
5.12 Voir les avis sur un artisan et laisser un avis (espace citoyen) . . . 55
5.13 Suivi des offres de service(espace citoyen) 56
5.14 Echanger via messagerie et profil (espace citoyen) 57
5.15 Tableau de bord et soumission d'une offre (espace artisan) 58
5.16 Gestion des offres de service et profil professionnel (espace artisan) 59
5.17 Tableau de bord et gestion des spécialités et lieux (espace administrateur) 60
5.18 Activer/ Désactiver les comptes(espace administrateur) 61
19 Modèle conceptuel des données de l'application Maalem 66

Introduction Générale **10**

Chapitre 1 : Contexte du projet **12**

1.1 Introduction 12

1.2 Cadrage du projet 12

1.2.1 Présentation du projet 12

1.2.2 Problématique 13

1.2.3 Étude des solutions existantes 14

1.2.4 Objectifs du projet 15

1.3 Planification du projet 16

1.3.1 Cycle de vie du projet 16

1.3.2 Planification : diagramme de Gantt 17

1.4 Conclusion 17

Chapitre 2 : Étude des besoins et analyse **19**

2.1 Introduction 19

2.2 Identification des acteurs 19

2.3 Spécification des besoins fonctionnels 20

2.4 Spécification des besoins non fonctionnels 22

2.5 Diagrammes de cas d'utilisation 23

2.5.1 Diagramme de cas d'utilisation global 23

2.5.2 Description textuelle des cas d'utilisation 25

2.6 Conclusion 28

Chapitre 3 : Conception et architecture **30**

3.1 Introduction 30

3.2 Conception 30
3.2.1 Diagramme de séquence 30
Postuler à une demande de service 30
3.2.2 Modèle conceptuel des données 31
Un stockage NoSQL plutôt que relationnel 31
3.3 Architecture 32
3.3.1 Architecture physique 33
3.3.2 Architecture logicielle 33
3.4 Conclusion 35

Chapitre 4 : Modèle d'IA de classification des spécialités 36
4.5 Introduction 36
4.6 Démarche générale 36
4.7 Constitution du jeu de données 37
4.8 Prétraitement des données 40
4.9 Architectures comparées 41
4.10 Métriques d'évaluation 42
4.11 Résultats et comparaison des architectures 43
4.12 Choix du modèle et affinage 44
4.12.1 Choix du seuil de confiance 47
4.13 Exportation pour le déploiement embarqué 47
4.14 Conclusion 48

Chapitre 5 : Réalisation 49
5.1 Outils utilisés 49
5.1.1 Environnement de développement 49
5.1.2 Outils de conception 50
5.1.3 Développement mobile 50
5.1.4 Développement web 51
5.1.5 Backend et base de données 51
5.1.6 Intelligence artificielle 51

5.1.7 Gestion de versions 52
5.2 Présentation des interfaces 52
5.2.1 Interfaces communes : authentification 52
5.2.2 Interfaces du citoyen 53
5.2.3 Interfaces de l'artisan 57
5.2.4 Interfaces de l'administrateur 59
5.3 Conclusion 61

Conclusion générale **62**

Bibliographie **64**

Annexe **65**
.1 Modèle conceptuel des données 65

Introduction Générale

Le secteur artisanal occupe une place centrale dans l'économie et la culture marocaines. Avec plus de deux millions d'artisans actifs à travers le pays, ce domaine représente une source majeure d'emploi et de revenus. Pourtant, la mise en relation entre les citoyens ayant besoin de services artisanaux et les artisans qualifiés demeure un processus largement informel, reposant essentiellement sur le bouche-à-oreille. Il en résulte plusieurs limites : une absence de traçabilité et de garantie de qualité, une visibilité réduite pour de nombreux artisans compétents, une perte de temps considérable pour le citoyen, ainsi qu'un manque de canal de communication structuré. S'ajoute à cela la difficulté, pour un citoyen non spécialisé, de décrire correctement la nature de la panne et de l'orienter vers le bon corps de métier.

Face à cette problématique, le projet **Maalem** propose le développement d'une application mobile visant à structurer, sécuriser et moderniser cette mise en relation. La plateforme réunit trois profils complémentaires — le citoyen, qui publie ses demandes de service ; l'artisan, qui soumet ses offres de service et réalise les missions ; et l'administrateur, qui supervise l'ensemble. L'apport le plus distinctif du projet réside dans l'intégration d'un modèle d'intelligence artificielle embarqué directement sur l'appareil mobile, capable de classifier automatiquement les types de pannes domestiques à partir d'une simple photo, simplifiant ainsi considérablement l'expression du besoin du citoyen.

Les objectifs poursuivis par ce projet sont les suivants : concevoir et développer une application mobile facilitant la mise en relation entre citoyens et artisans ; simplifier l'expression des besoins grâce à la classification automatique des pannes par IA ; offrir aux artisans une vitrine professionnelle numérique augmentant leur visibilité et leurs opportunités ; garantir une communication fluide et un suivi transparent tout au long de la prestation ; et fournir une interface d'administration pour la supervision et la modération de la plateforme.

Pour répondre à ces objectifs, le présent rapport est structuré en cinq chapitres :

- **Chapitre 1 — Contexte du projet :** présentation du projet,problématique, étude des solutions existantes, objectifs et planification du projet.
- **Chapitre 2 — Analyse des besoins :** identification des acteurs, besoins fonctionnels et non fonctionnels, et diagrammes de cas d'utilisation.
- **Chapitre 3 — Conception et architecture :** diagramme de séquence, diagramme de classes et choix architecturaux.
- **Chapitre 4 — Modèle d'IA de classification des spécialités :** données, comparaison des architectures, entraînement, évaluation et export TensorFlow Lite.
- **Chapitre 5 — Réalisation :** outils utilisés et interfaces développées.

Chapitre 1

Contexte du projet

1.1 Introduction

C'est en observant autour de nous les difficultés concrètes rencontrées au quotidien pour trouver un artisan fiable — que ce soit pour une fuite d'eau, une panne électrique ou un problème de menuiserie — que l'idée de Maalem a pris forme. Ce vécu, couplé au constat d'un secteur encore peu digitalisé au Maroc, nous a convaincues de la pertinence et de l'utilité d'un tel projet.

Ce premier chapitre présente d'abord le projet Maalem dans ses grandes lignes, puis expose la problématique centrale à laquelle il répond. Il établit ensuite une étude comparative des solutions existantes, aussi bien au Maroc qu'à l'international, avant de définir les objectifs poursuivis et la planification du projet.

1.2 Cadrage du projet

1.2.1 Présentation du projet

Maalem est une application mobile dont l'objectif est de structurer, sécuriser et moderniser la mise en relation entre les citoyens ayant besoin de services artisanaux et les artisans qualifiés au Maroc. Son nom, qui désigne en arabe dialectal marocain l'artisan maître dans son métier, reflète directement la vocation de la plateforme.

L'application s'articule autour de trois profils complémentaires. Le **citoyen** publie des demandes de service, reçoit et compare les offres des artisans, puis choisit librement celui qui lui convient. L'**artisan** consulte les demandes correspondant à sa spécialité et à sa zone géographique, soumet des offres personnalisées et réalise les missions. L'**administrateur** supervise l'ensemble de la plateforme via une interface web dédiée : validation des artisans, gestion des utilisateurs, modération et statistiques.

L'apport le plus distinctif du projet réside dans l'intégration d'un modèle d'intelligence artificielle embarqué directement sur l'appareil mobile, capable de classifier automatiquement la nature des pannes domestiques à partir d'une simple photo. Cette fonctionnalité simplifie considérablement l'expression du besoin pour les citoyens peu familiers avec les terminologies techniques et permet un meilleur ciblage des artisans qualifiés.

1.2.2 Problématique

Au Maroc, trouver un artisan fiable pour des travaux de réparation domestique constitue un défi quotidien. Le processus actuel repose sur le bouche-à-oreille, avec plusieurs limites :

- **Absence de traçabilité :** il est difficile d'évaluer la compétence réelle de l'artisan avant l'intervention.
- **Visibilité limitée des artisans :** de nombreux artisans compétents peinent à trouver des clients au-delà de leur cercle de connaissances.
- **Perte de temps :** le citoyen doit multiplier les appels et déplacements pour trouver un artisan disponible et compétent dans sa zone.
- **Difficulté à décrire le problème :** les citoyens non spécialisés peinent à identifier et formuler correctement la nature de la panne (peinture, électricité, maçonnerie...).
- **Absence de canal de communication structuré :** les échanges se font par téléphone ou en personne, sans historique ni suivi de la conversation.

La problématique centrale est donc : *comment concevoir une plateforme*

numérique mobile qui facilite, structure et sécurise la mise en relation entre citoyens et artisans au Maroc, tout en exploitant l'intelligence artificielle pour simplifier l'expression des besoins ?

1.2.3 Étude des solutions existantes

Avant de concevoir Maalem, une étude comparative des solutions existantes a été menée, aussi bien au Maroc qu'à l'international :

- Bricole [1]

Bricole est une application mobile marocaine disponible sur Google Play. Elle permet aux citoyens de rechercher des artisans par ville, et de les contacter directement par téléphone. Cependant, elle ne propose pas de système d'offres structuré, pas de chat intégré, pas de classification IA des problèmes.

- Saweblia [2]

Saweblia est un service marocain de dépannage et réparation à domicile. L'entreprise se distingue par une vérification rigoureuse des artisans et une garantie de 6 mois sur les prestations. Néanmoins, Saweblia fonctionne comme une agence intermédiaire — le client ne choisit pas directement son artisan — ne dispose pas d'application mobile et n'utilise pas d'IA.

- TaskRabbit [3]

TaskRabbit est une plateforme américaine de mise en relation entre particuliers et prestataires de services à domicile, active dans plusieurs pays (États-Unis, Royaume-Uni, Canada, Allemagne, France). Elle propose un système d'offres structuré, des profils détaillés avec avis et tarifs horaires, ainsi qu'un chat intégré. En revanche, elle ne dispose d'aucune composante d'IA pour la classification des problèmes, et son modèle tarifaire (abonnement mensuel pour les prestataires) la rend peu adaptée au contexte marocain.

- Tableau comparatif

Le tableau 1.1 synthétise le positionnement de chaque solution selon les critères clés que nous avons identifiés.

TABLE 1.1 – Comparaison des solutions existantes

Critère	**Bricole**	**Saweblia**	**TaskRabbit**	**Maalem**
Application mobile	Oui	Non	Oui	Oui
Système d'offres structuré	Non	Non	Oui	Oui
Chat en temps réel	Non	Non	Oui	Oui
Vérification des artisans	Limitée	Rigoureuse	Oui	Oui (CIN)
Choix libre de l'artisan	Oui	Non	Oui	Oui
Classification par IA	Non	Non	Non	Oui
Adapté au contexte marocain	Oui	Oui	Non	Oui

- Résultat

L'analyse de ces solutions révèle qu'aucune ne combine simultanément une application mobile, un système d'offres structuré, un chat en temps réel et une composante d'intelligence artificielle pour la classification automatique des problèmes. De plus, les plateformes marocaines existantes fonctionnent soit comme des annuaires passifs, soit comme des agences intermédiaires, sans laisser au citoyen la liberté de choisir son artisan. C'est précisément ce vide que Maalem entend combler.

1.2.4 Objectifs du projet

Le projet poursuit deux grands axes d'objectifs complémentaires : un axe applicatif, centré sur la plateforme de mise en relation, et un axe intelligence artificielle, centré sur la simplification de l'expression du besoin.

- Objectif 1 : la plateforme Maalem

Concevoir et développer une application mobile complète facilitant la mise en

relation entre citoyens et artisans au Maroc. Ce premier axe vise plus précisément à :

- offrir aux artisans une vitrine professionnelle numérique augmentant leur visibilité et leurs opportunités ;
- garantir une communication fluide et un suivi transparent tout au long du processus de prestation, grâce à un système d'offres, une messagerie en temps réel et un système d'évaluation ;
- fournir une interface d'administration web pour la supervision, la validation des artisans et la modération de la plateforme.

- Objectif 2 : le modèle IA de classification des spécialités

Simplifier l'expression des besoins du citoyen grâce à un modèle d'intelligence artificielle embarqué, capable de classifier automatiquement les pannes domestiques à partir de photos. Ce second axe vise à :

- concevoir un modèle léger et performant, déployable directement sur l'appareil via TensorFlow Lite ;
- pré-remplir automatiquement la catégorie de la demande à partir d'une simple photo, sans exiger de l'utilisateur une connaissance technique préalable ;
- orienter efficacement chaque demande vers la spécialité d'artisan appropriée, améliorant ainsi la pertinence du matching.

1.3 Planification du projet

1.3.1 Cycle de vie du projet

Le choix d'une méthodologie de développement adaptée est déterminant pour la réussite d'un projet logiciel. Pour la réalisation de Maalem, nous avons opté pour un **cycle de vie incrémental et itératif**.

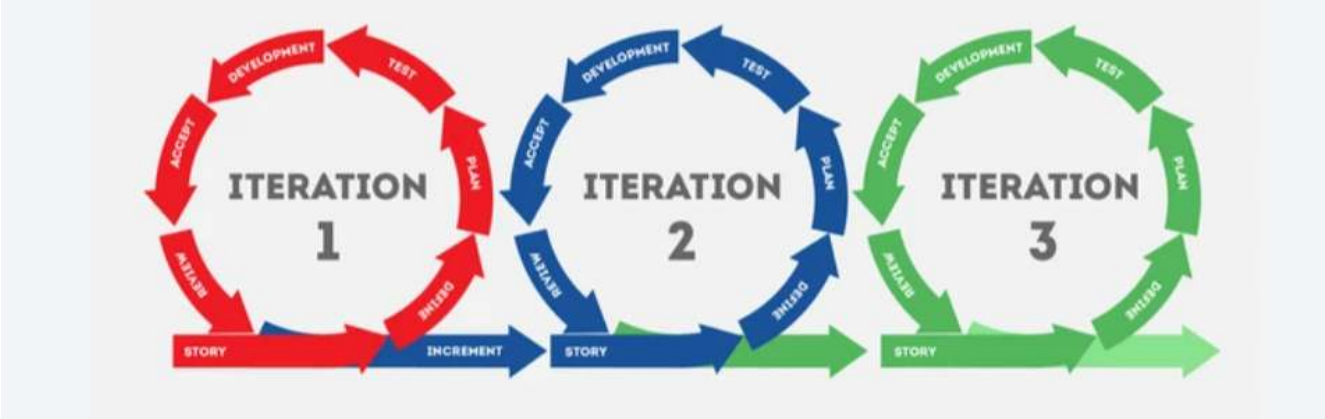


FIGURE 1.1 – Cycle de vie de maalem

1.3.2 Planification : diagramme de Gantt

Afin d'organiser et de suivre le déroulement des différentes phases du projet dans le temps, nous avons établi un diagramme de Gantt.

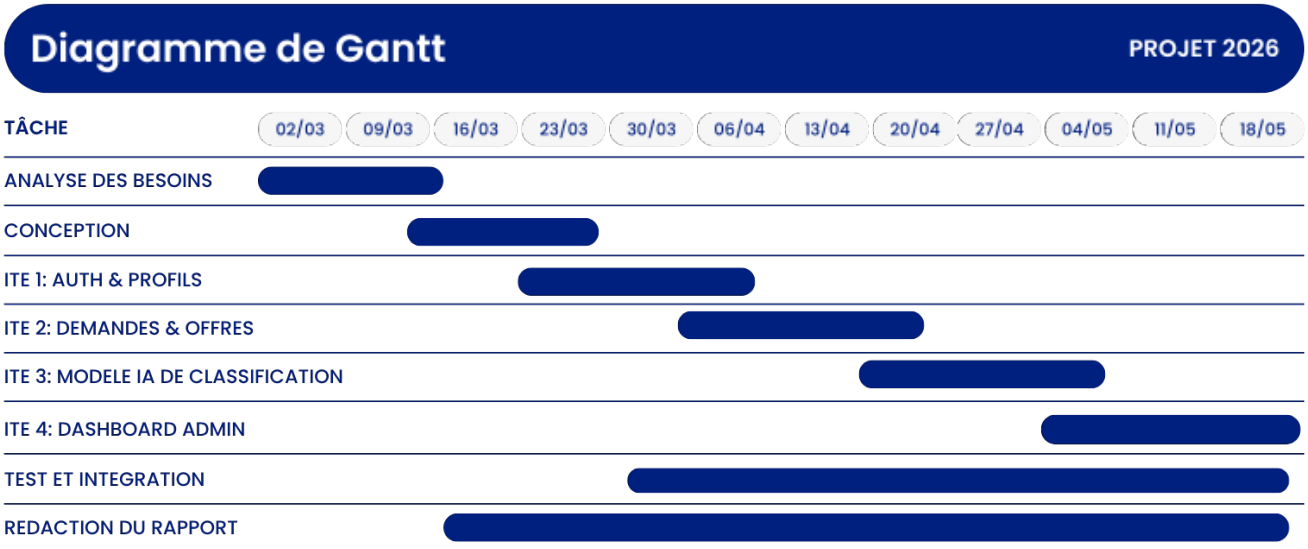


FIGURE 1.2 – Diagramme de Gantt du projet Maalem

1.4 Conclusion

Ce premier chapitre a permis de poser les fondations du projet en présentant l'application Maalem, en définissant clairement la problématique et en position-

nant la solution par rapport aux offres existantes à travers une étude comparative. Il a également structuré les objectifs autour de deux axes complémentaires : la plateforme de mise en relation et le modèle d'intelligence artificielle. Le chapitre suivant sera consacré à l'analyse détaillée des besoins fonctionnels et non fonctionnels.

Chapitre 2

Étude des besoins et analyse

2.1 Introduction

Ce chapitre est consacré à l'identification et à la spécification des besoins du système Maalem. Il présente d'abord le cycle de vie adopté pour le développement du projet, puis les acteurs, les besoins fonctionnels et non fonctionnels, et enfin les diagrammes de cas d'utilisation modélisant les interactions entre les acteurs et le système.

2.2 Identification des acteurs

En ce qui suit, une identification des acteurs principaux qui interagissent directement avec notre application.

Administrateur : il s'agit de l'acteur qui supervise la plateforme à travers une interface web dédiée. Il est responsable de la validation des inscriptions des artisans en vérifiant l'authenticité de leur pièce d'identité (CIN), de la gestion des spécialités de services proposés et les villes d'intérvention, ainsi que de la modération générale de la plateforme. Il dispose également d'un tableau de bord statistique lui permettant de suivre l'activité globale de l'application et d'envoyer des notifications globales aux utilisateurs en cas de besoin.

Citoyen : il s'agit d'un utilisateur de l'application mobile qui cherche à bénéficier de services artisanaux pour la réparation ou l'entretien de son domicile. Après inscription, le citoyen peut publier des demandes de services en remplissant manuellement le formulaire ou en téléchargeant simplement une photo du problème, traitée par le modèle d'intelligence artificielle qui pré-remplit automatiquement la catégorie de service correspondante. Il reçoit ensuite les offres des artisans intéressés, peut consulter leurs profils détaillés (portfolio, avis, note moyenne), comparer les propositions et choisir librement l'artisan qui lui convient. Le citoyen peut également communiquer avec l'artisan retenu via le chat intégré, suivre l'avancement de sa mission, et évaluer la prestation à la fin du service en attribuant une note et un commentaire.

Artisan (Maalem) : il s'agit d'un professionnel qualifié dans un domaine artisanal spécifique (plomberie, électricité, maçonnerie, menuiserie, etc.) qui utilise l'application pour proposer ses services et trouver de nouvelles missions. Après création de son profil professionnel (spécialité, zone géographique d'intervention, portfolio de réalisations, pièce d'identité) et validation par l'administrateur, l'artisan peut consulter les demandes de service disponibles dans sa zone et correspondant à sa spécialité. Il a la possibilité d'envoyer des offres de service personnalisées incluant le prix proposé, le délai d'exécution et un message descriptif. Une fois une offre acceptée, il communique avec le client via le chat intégré, réalise la mission convenue.

2.3 Spécification des besoins fonctionnels

L'application Maalem sera développée pour fournir les fonctionnalités suivantes, organisées selon les acteurs principaux du système.

Gestion des comptes et authentification : la plateforme repose sur un

système d'authentification sécurisé Firebase Authentication, garantissant l'identification fiable de chaque utilisateur. Après l'inscription initiale, chaque utilisateur dispose d'un espace personnel où il peut compléter son profil, mettre à jour ses informations et personnaliser les paramètres de son compte. Pour les artisans, l'inscription comprend une étape supplémentaire de validation administrative basée sur la vérification de la CIN, afin d'assurer la fiabilité et la confiance au sein de la plateforme.

Publication et classification intelligente des demandes de service : l'une des fonctionnalités innovantes de Maalem repose sur l'intégration d'un modèle d'intelligence artificielle déployé localement via TensorFlow Lite. Lorsque le citoyen souhaite publier une demande, il peut simplement prendre une photo du problème, et le modèle classifie automatiquement la nature du problème pour préremplir la catégorie de service appropriée. Cette approche simplifie considérablement le processus de création de demande, notamment pour les utilisateurs peu familiers avec les terminologies techniques, et permet un meilleur ciblage des artisans qualifiés.

Système d'offres de service et de matching : après la publication d'une demande de service, les artisans correspondant à la catégorie et à la zone géographique concernées sont notifiés et peuvent consulter les détails de la demande de service. Chaque artisan intéressé soumet une offre de service comprenant le prix proposé, le délai d'exécution et un message décrivant son approche. Le citoyen reçoit l'ensemble des offres reçues et peut les comparer en consultant les profils détaillés des artisans, notamment leurs évaluations antérieures et leur note moyenne. Cette transparence permet au citoyen de faire un choix éclairé selon ses propres critères (prix, délai, expérience).

Système de messagerie en temps réel : une fois qu'une offre de service est acceptée, un canal de communication direct s'ouvre entre le citoyen et l'artisan

via le chat intégré. Ce système de messagerie, basé sur Firebase Firestore, permet d'échanger en temps réel pour clarifier les détails de la mission, partager des informations complémentaires et organiser l'intervention. Le chat constitue également une trace écrite utile en cas de désaccord ultérieur, permettant à l'administrateur de consulter l'historique des échanges lors du traitement d'une éventuelle réclamation.

Évaluation et système de réputation : à la fin de chaque prestation, le citoyen est invité à évaluer le travail de l'artisan en lui attribuant une note et en rédigeant un commentaire. Ces évaluations s'agrègent dans le profil de l'artisan sous forme de note moyenne et d'avis détaillés, contribuant à construire sa réputation sur la plateforme. Ce système de feedback joue un rôle essentiel pour orienter les futurs choix des citoyens et inciter les artisans à fournir un travail de qualité.

Tableau de bord administrateur : l'administrateur dispose d'une interface web et mobile dédiées comprenant un tableau de bord statistique offrant une vue globale de l'activité de la plateforme (nombre d'utilisateurs, demandes publiées, missions réalisées, etc.). Depuis cette interface, il peut gérer les utilisateurs, valider les inscriptions des artisans, administrer les spécialités de services,et envoyer des notifications globales pour informer la communauté d'éventuelles évolutions ou opérations.

2.4 Spécification des besoins non fonctionnels

Pour assurer le bon fonctionnement de l'application mobile et garantir la satisfaction des utilisateurs, des contraintes doivent être prises en compte tout au long de l'élaboration du projet :

— **Ergonomie et convivialité :** l'application doit fournir une interface homme-machine (IHM) simple et facile à utiliser pour une navigation intuitive.
— **Disponibilité :** backend serverless Firebase garantissant une haute disponibilité avec synchronisation offline.
— **Scalabilité :** architecture Firestore NoSQL permettant une montée en charge progressive.
— **Rapidité et robustesse :** l'application doit garantir la réactivité et la fiabilité des résultats.
— **Sécurité :** authentification par email et par rôle, données protégées par Firebase, accès admin restreint.
— **Modularité :** système conçu pour permettre l'ajout de nouvelles fonctionnalités à l'avenir.

2.5 Diagrammes de cas d'utilisation

Le diagramme de cas d'utilisation est un diagramme UML qui permet de représenter de manière graphique les interactions entre les acteurs et le système. Il offre une vision globale et synthétique des fonctionnalités offertes par l'application Maalem, en mettant en évidence les services attendus par chaque acteur.

2.5.1 Diagramme de cas d'utilisation global

Le diagramme suivant présente une vue d'ensemble des cas d'utilisation de l'application Maalem.

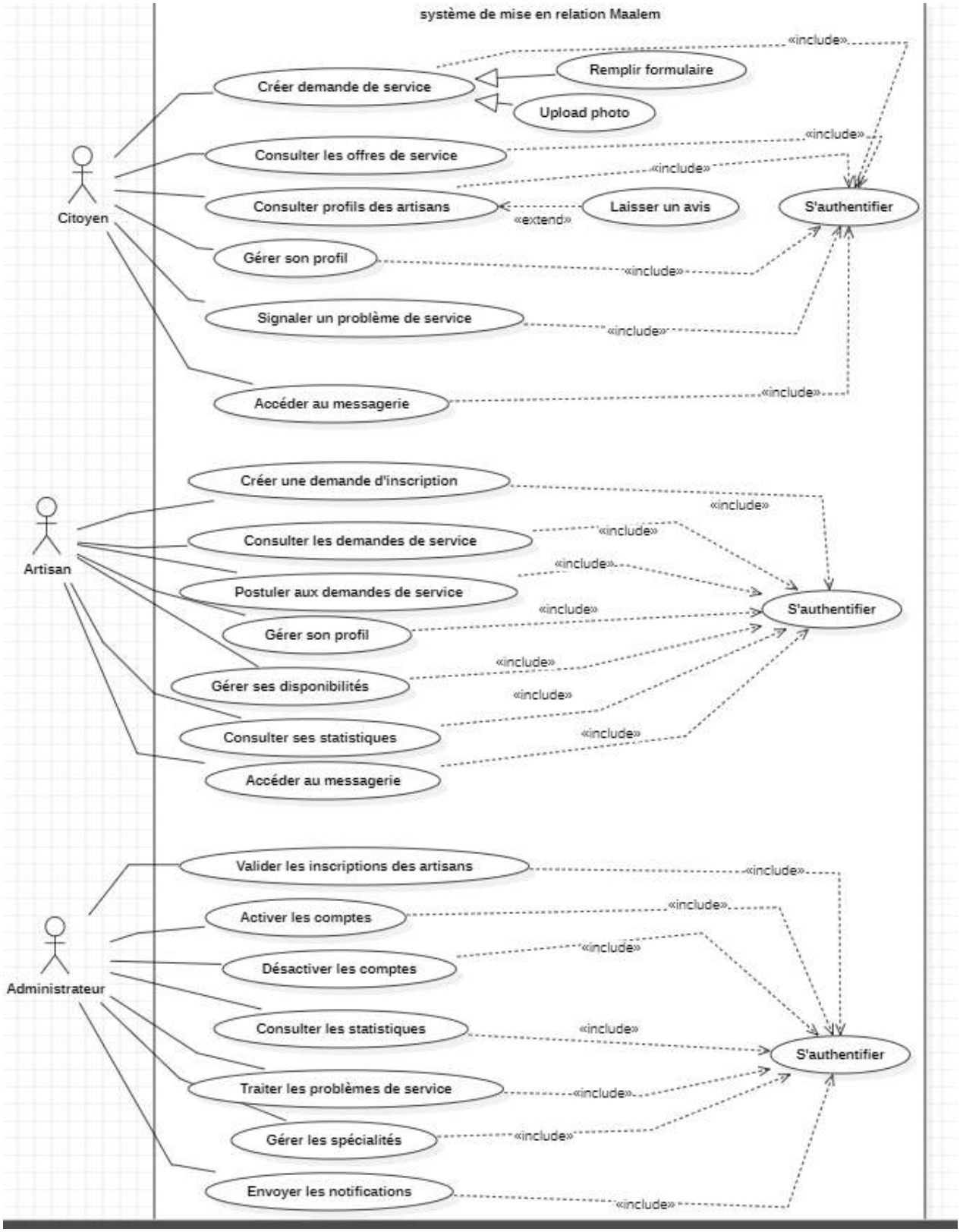


FIGURE 2.3 – Diagramme des cas d'utilisations global

2.5.2 Description textuelle des cas d'utilisation

Afin de préciser le déroulement des principaux cas d'utilisation, nous présentons ci-dessous leurs descriptions textuelles détaillées sous forme de tableaux. Nous avons retenu quatre cas représentatifs couvrant l'ensemble des acteurs du système.

TABLE 2.2 – Description du cas d'utilisation « Publier une demande de service »

Cas d'utilisation	Publier une demande de service
Acteur principal	Citoyen
Description	Permet au citoyen de publier une demande de service, soit en remplissant le formulaire manuellement, soit en téléchargeant une photo classifiée automatiquement par le modèle d'IA.
Pré-condition	Le citoyen est authentifié sur l'application.
Scénario nominal	1. Le citoyen accède au formulaire de création de demande. 2. Il prend ou téléverse une photo du problème. 3. Le modèle d'IA classifie l'image et pré-remplit la catégorie. 4. Le citoyen complète les informations (titre, description, ville). 5. Il valide la publication. 6. Le système enregistre la demande et notifie les artisans concernés.
Scénario alternatif	Si la confiance de l'IA est inférieure au seuil, le citoyen sélectionne lui-même la catégorie.
Post-condition	La demande est publiée et visible par les artisans correspondants.

TABLE 2.3 – Description du cas d'utilisation « Postuler à une demande de service »

Cas d'utilisation	Postuler à une demande de service
Acteur principal	Artisan
Description	Permet à un artisan de consulter les demandes disponibles dans sa zone et sa spécialité, puis de soumettre une offre de service.
Pré-condition	L'artisan est authentifié et son inscription a été validée par l'administrateur.
Scénario nominal	1. L'artisan consulte la liste des demandes correspondant à sa spécialité et sa zone. 2. Il sélectionne une demande et consulte ses détails. 3. Il saisit son offre (prix, délai, message descriptif). 4. Il valide et soumet l'offre. 5. Le système enregistre l'offre et notifie le citoyen.
Scénario alternatif	Si les champs obligatoires sont incomplets, le système affiche un message d'erreur.
Post-condition	L'offre est enregistrée et associée à la demande concernée.

TABLE 2.4 – Description du cas d'utilisation « Évaluer une prestation »

Cas d'utilisation	Évaluer une prestation
Acteur principal	Citoyen
Description	Permet au citoyen d'attribuer une note et un commentaire à l'artisan après la réalisation d'une mission.
Pré-condition	La mission est terminée et le citoyen est authentifié.
Scénario nominal	1. Le citoyen accède à la mission terminée. 2. Il attribue une note et rédige un commentaire. 3. Il valide son évaluation. 4. Le système enregistre l'avis et recalcule la note moyenne de l'artisan.
Scénario alternatif	Le citoyen peut choisir de ne pas laisser de commentaire et de ne fournir qu'une note.
Post-condition	L'avis est ajouté au profil de l'artisan et sa réputation est mise à jour.

TABLE 2.5 – Description du cas d'utilisation « Valider l'inscription d'un artisan »

Cas d'utilisation	Valider l'inscription d'un artisan
Acteur principal	Administrateur
Description	Permet à l'administrateur de vérifier les informations d'un artisan en attente et de valider ou refuser son inscription.
Pré-condition	L'administrateur est authentifié et au moins un artisan est en attente de validation.
Scénario nominal	1. L'administrateur accède à la liste des artisans en attente. 2. Il sélectionne un artisan et examine son dossier (spécialité, zone, CIN). 3. Il vérifie l'authenticité des informations. 4. Il valide l'inscription. 5. Le système met à jour le statut de l'artisan à « validé ».
Scénario alternatif	En cas d'informations non conformes, l'administrateur refuse l'inscription et l'artisan en est informé.
Post-condition	L'artisan validé accède aux fonctionnalités de la plateforme.

2.6 Conclusion

Ce deuxième chapitre nous a permis d'explorer en profondeur les besoins spécifiques des utilisateurs et des parties prenantes de l'application mobile. Après avoir présenté le cycle de vie incrémental et itératif retenu ainsi que sa planification, nous avons identifié les acteurs, spécifié les besoins fonctionnels et non fonctionnels, puis modélisé les interactions à travers les diagrammes de cas d'utilisation et leurs descriptions textuelles détaillées.

En se concentrant sur l'étude approfondie des besoins, cette phase revêt une importance cruciale pour la conception d'une solution répondant à toutes les at-

tentes et exigences des utilisateurs. Dans le prochain chapitre, nous aborderons l'analyse et la conception, en nous concentrant sur la modélisation UML et la conception architecturale.

Chapitre 3

Conception et architecture

3.1 Introduction

Dans ce chapitre, nous nous intéressons à la conception du système Maalem, puis à son architecture. Nous commençons par la modélisation UML : les diagrammes de séquence pour les scénarios clés et le modèle conceptuel des données. Nous présentons ensuite l'architecture du système sous ses deux aspects, physique et logique.

3.2 Conception

3.2.1 Diagramme de séquence

Postuler à une demande de service

Ce diagramme de séquence illustre le processus par lequel un artisan consulte les demandes de service disponibles et soumet une offre de service à un citoyen.

Description des interactions :

— **Consultation des demandes :** L'artisan accède à la liste des demandes de service disponibles correspondant à sa spécialité et à sa zone géographique.

— **Affichage des demandes :** La liste des demandes présente à l'artisan l'ensemble des demandes filtrées selon son métier.

— **Sélection d'une demande de service :** L'artisan sélectionne une demande, consulte ses détails et choisit d'y postuler.
— **Saisie de l'offre de service :** L'artisan renseigne les informations de son offre : le prix proposé, le délai d'exécution et un message descriptif.
— **Soumission et association :** L'offre de service ainsi créée est soumise puis associée à la demande concernée.
— **Mise à jour de l'état :** La demande de service passe à l'état « Déjà postulé » afin d'éviter toute candidature en double de la part de l'artisan.
— **Confirmation :** L'interface affiche le bouton « Déjà postulé », confirmant à l'artisan que son offre a bien été soumise.

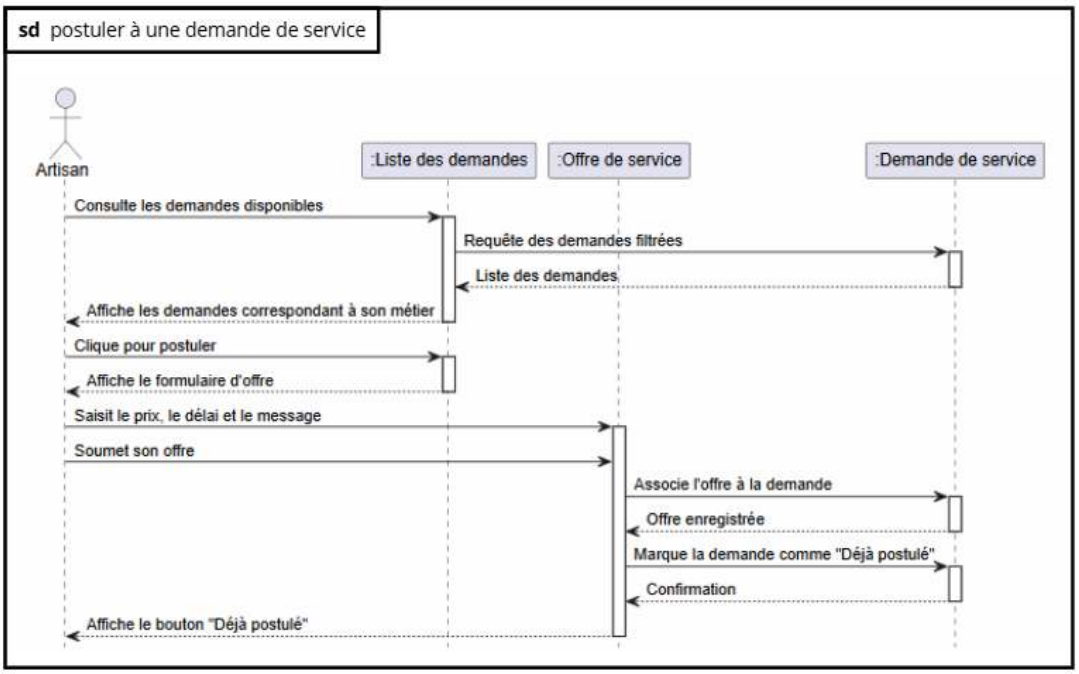


FIGURE 3.4 – Diagramme de séquence : Postuler à une demande de service

3.2.2 Modèle conceptuel des données

Un stockage NoSQL plutôt que relationnel

À l'implémentation, Maalem ne repose pas sur une base de données relationnelle mais sur **Cloud Firestore**, une base de données **NoSQL orientée documents**. Contrairement à une base relationnelle, qui organise les données en tables

liées par des clés étrangères, Firestore structure les données en *collections* contenant des *documents* flexibles. Les entités métier de l'application (**User**, **Citoyen**, **Artisan**, **Demande de service**, **Offre de service**, **Messagerie**, **Avis**, etc.) se traduisent donc, dans l'application, par des collections de documents et non par des tables. Une représentation conceptuelle de ces entités, de leurs attributs et de leurs relations est fournie en annexe (voir l'annexe .1).

Ce choix d'un stockage NoSQL se justifie par plusieurs raisons adaptées à notre contexte :

- **Flexibilité du schéma :** les documents ne sont pas contraints par un schéma rigide, ce qui facilite l'évolution du modèle de données et l'ajout de nouvelles fonctionnalités sans migration lourde.
- **Adéquation avec le temps réel :** le modèle documentaire de Firestore est conçu nativement pour la synchronisation en temps réel, essentielle pour la messagerie et la mise à jour des offres, là où une base relationnelle classique nécessiterait des mécanismes supplémentaires.
- **Performance en lecture :** la structure orientée documents permet de récupérer rapidement un ensemble d'informations liées sans recourir à des jointures coûteuses.
- **Scalabilité horizontale :** les bases NoSQL comme Firestore sont conçues pour monter en charge et supporter un grand nombre d'utilisateurs simultanés.

3.3 Architecture

Le choix d'une architecture adaptée est déterminant pour garantir la maintenabilité, la testabilité et l'évolutivité de l'application. Nous présentons dans cette section l'architecture du système Maalem sous deux angles complémentaires : l'architecture physique, qui décrit le déploiement des composants et leurs communications, puis l'architecture logicielle, qui décrit l'organisation interne du code.

3.3.1 Architecture physique

L'architecture physique décrit le déploiement réel des différents composants de l'application Maalem ainsi que les protocoles de communication qui les relient. La solution repose sur une architecture client-serveur sans serveur dédié (*serverless*), s'appuyant sur les services cloud de Firebase. Le système se compose des éléments suivants :

- **Le client mobile :** l'application native Android (smartphones et tablettes), destinée aux citoyens et aux artisans. C'est elle qui embarque le modèle d'IA via TensorFlow Lite.
- **Le client web :** l'interface d'administration développée en React.js, accessible depuis un navigateur, et utilisée par l'administrateur pour superviser la plateforme.
- **Le backend serverless (Firebase) :** il regroupe les services cloud de Google : Cloud Firestore pour le stockage des données en temps réel, Firebase Authentication, Firebase Storage pour le stockage des images, et Firebase Cloud Messaging (FCM) pour les notifications push.

La communication entre les clients et le backend s'effectue via le protocole sécurisé HTTPS. Les échanges en temps réel (chat, mise à jour des demandes et des offres) s'appuient sur les mécanismes de synchronisation de Firestore, tandis que les notifications sont acheminées par le service FCM.

3.3.2 Architecture logicielle

Pour le développement de l'application mobile Maalem, nous avons adopté le patron architectural **MVVM (Model-View-ViewModel)**, en nous inspirant des principes de la **Clean Architecture**.

Le patron MVVM :

Le patron MVVM organise le code en trois couches distinctes qui séparent clairement la logique de présentation de la logique métier et de l'interface utilisateur :

— **La couche View :** elle correspond à l'interface utilisateur (les écrans réalisés en Kotlin avec des fichiers XML de layout). Son rôle est uniquement d'afficher les données et de capter les interactions de l'utilisateur, sans contenir de logique métier.

— **La couche ViewModel :** elle fait l'intermédiaire entre la View et le Model. Elle expose les données observables consommées par l'interface et gère la logique de présentation, tout en survivant aux changements de configuration (rotation de l'écran, etc.).

— **La couche Model :** elle représente les données de l'application et la logique d'accès à ces données (interactions avec Firebase Firestore, Authentication et Storage).

L'intérêt majeur du MVVM réside dans la séparation des responsabilités et la liaison de données (*data binding*) : la View observe les changements du ViewModel et se met à jour automatiquement, ce qui réduit le couplage et facilite les tests. Le schéma suivant illustre ce flux entre les trois couches.

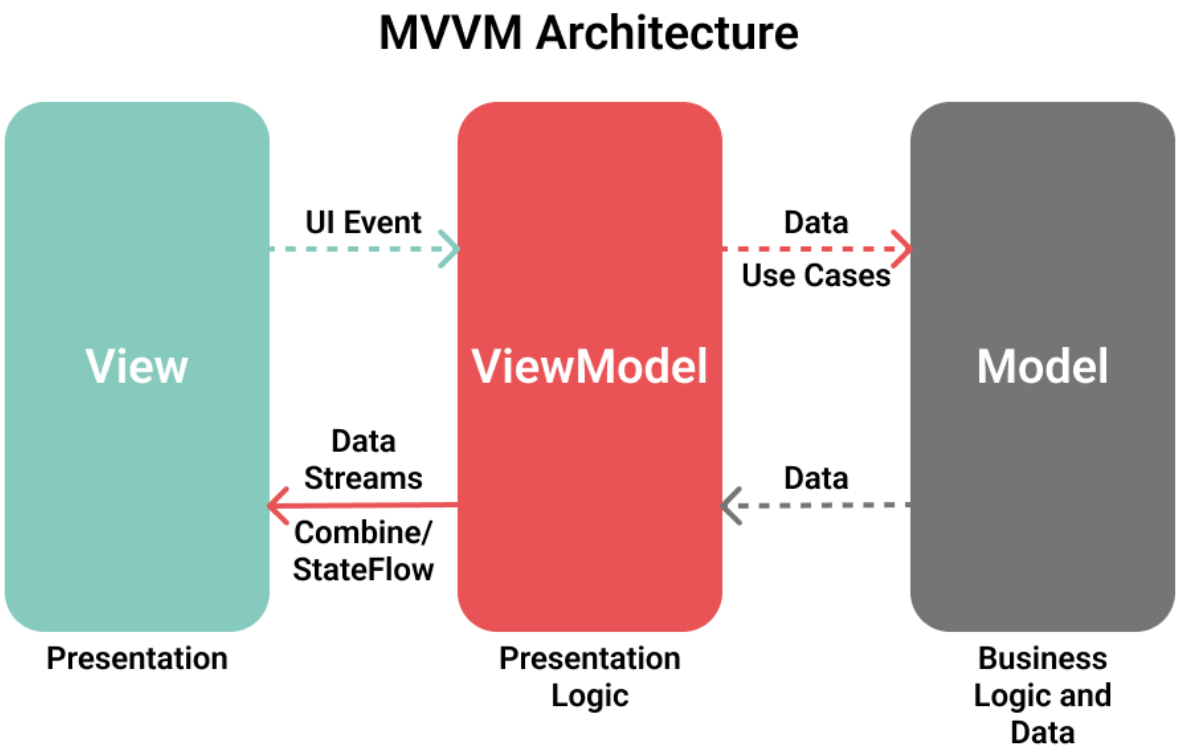


FIGURE 3.5 – Le patron architectural MVVM

Inspiration de la Clean Architecture :

Afin de renforcer la robustesse et la séparation des préoccupations, nous nous sommes inspirées des principes de la Clean Architecture. Cette approche préconise une organisation du code en couches concentriques où les dépendances sont orientées vers l'intérieur, c'est-à-dire des détails techniques vers la logique métier. Concrètement, nous avons structuré l'application autour de plusieurs couches :

- **La couche de présentation (Presentation) :** regroupe les Views et les ViewModels, responsables de l'affichage et de la gestion des interactions.
- **La couche domaine (Domain) :** contient la logique métier pure de l'application, indépendante de tout framework, sous forme de cas d'utilisation (use cases) et d'entités.
- **La couche données (Data) :** implémente l'accès aux sources de données externes (Firebase) à travers des repositories qui exposent des interfaces abstraites au domaine.

Cette séparation en couches, combinée à l'injection de dépendances via **Hilt**, permet de découpler les composants, de faciliter les tests unitaires et de rendre le code plus modulaire et évolutif. Ainsi, une modification de la source de données ou de la technologie de stockage n'impacte pas la logique métier de l'application.

3.4 Conclusion

Dans ce chapitre, nous nous sommes concentrées sur les aspects conceptuels et architecturaux de notre projet. Nous avons d'abord fourni une conception détaillée à travers des diagrammes UML — diagrammes de séquence et modèle conceptuel des données — définissant les vues dynamiques et statiques de la plateforme, tout en précisant le recours à un stockage NoSQL. Nous avons ensuite présenté l'architecture du système sous ses aspects physique et logiciel, en détaillant le patron MVVM inspiré de la Clean Architecture.

Chapitre 4

Modèle d'IA de classification des spécialités

4.5 Introduction

L'un des apports distinctifs de la plateforme Maalem réside dans l'intégration d'un modèle d'intelligence artificielle embarqué directement sur l'appareil mobile. Ce modèle vise à simplifier l'expression du besoin pour le citoyen : à partir d'une simple photographie d'un défaut sur un mur, il identifie automatiquement la nature du problème et oriente la demande vers la spécialité d'artisan appropriée. Cette fonctionnalité répond à une difficulté concrète identifiée dès l'étude préalable, à savoir l'incapacité d'un utilisateur non spécialisé à décrire correctement la nature d'une panne et à la rattacher au bon corps de métier.

Ce chapitre présente la démarche expérimentale ayant conduit au modèle retenu. Il décrit successivement le jeu de données constitué, le prétraitement appliqué, les architectures comparées, les métriques d'évaluation, les résultats obtenus, ainsi que la justification du choix final et son exportation au format embarqué.

4.6 Démarche générale

La conception du modèle a suivi une démarche expérimentale et comparative, fondée sur le principe de l'apprentissage par transfert. Plutôt que d'entraîner un réseau de neurones à partir de zéro — ce qui exigerait un volume de données considérable et une puissance de calcul importante — nous avons réutilisé des

architectures pré-entraînées sur la base ImageNet, puis les avons adaptées à notre problème spécifique de classification des défauts de bâtiment.

La démarche s'articule autour des grandes étapes suivantes : constitution et nettoyage du jeu de données, prétraitement et découpage en ensembles d'entraînement, de validation et de test, comparaison de plusieurs architectures candidates selon des critères de précision et de contrainte embarquée, affinage du modèle le plus adapté, évaluation détaillée par classe, puis exportation au format TensorFlow Lite pour le déploiement sur mobile.

4.7 Constitution du jeu de données

Le jeu de données a été constitué à partir d'une source principale issue de la plateforme **Roboflow** (projet de classification d'images de défauts de bâtiment), complétée par un ensemble d'images hors-domaine destiné à renforcer la robustesse du modèle.

Les classes d'origine ont été regroupées en quatre catégories cohérentes avec les spécialités de l'application :

- **fissure** : regroupe les fissures majeures et mineures des murs, orientées vers la spécialité *Maçonnerie*.
- **peinture** : correspond aux défauts d'écaillage et de décollement de la peinture, orientés vers la spécialité *Peinture*.
- **humidite** : regroupe les signes d'humidité, à savoir les taches et les traces d'algues, orientés vers la spécialité *Étanchéité*. Ce regroupement constitue un compromis assumé, justifié par la similarité visuelle de ces manifestations d'infiltration.
- **autre** : classe de rejet regroupant les murs sains ainsi que des images génériques hors-domaine. Cette classe permet au modèle de reconnaître qu'une image ne correspond à aucun défaut traitable, plutôt que de la forcer arbitrairement vers une catégorie métier.

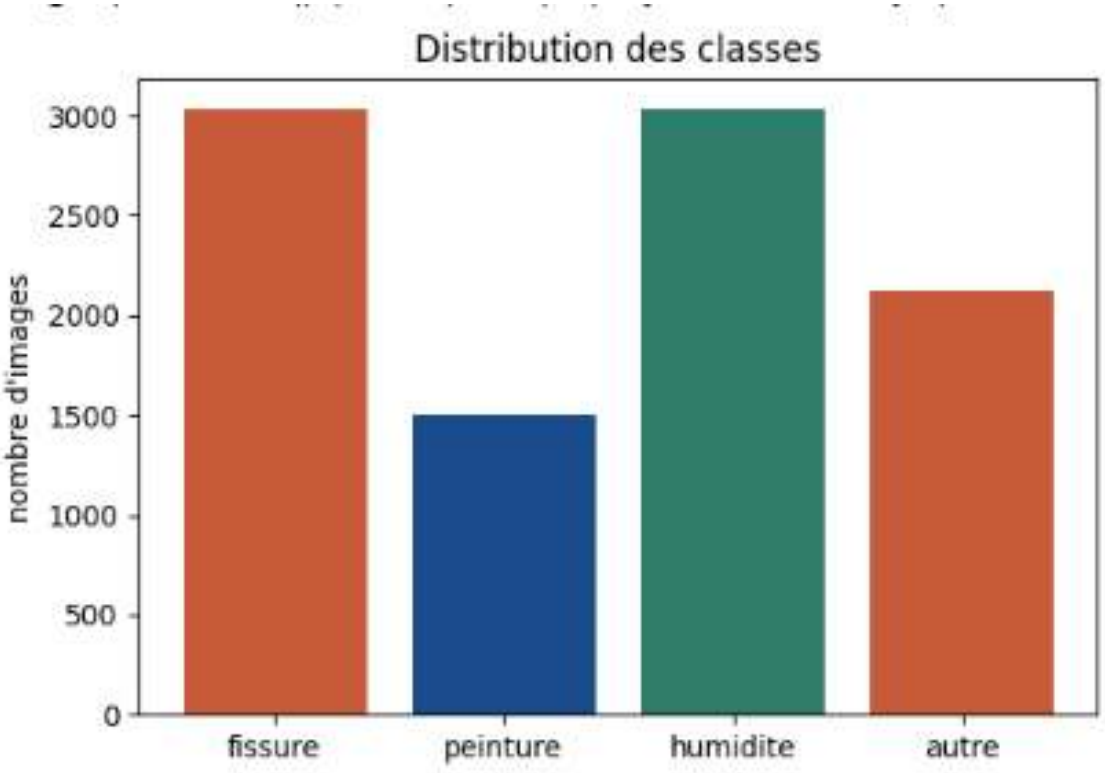


FIGURE 4.6 – Distribution des classes

L'introduction de la classe **autre** répond à une limite intrinsèque des classifieurs à fonction softmax, qui répartissent toujours la totalité de la probabilité entre les classes connues. Sans cette classe, une photographie sans rapport avec un mur serait inévitablement classée — avec une confiance souvent élevée — dans l'une des trois catégories métier. Les images hors-domaine ont été tirées du jeu de données public CIFAR-10 (objets, animaux, véhicules), de manière à apprendre au modèle à distinguer « ce qui n'est pas un défaut de mur ».



FIGURE 4.7 – Jeu de données par classe

Le tableau 4.6 présente la répartition finale des images entre les ensembles d'entraînement, de validation et de test.

TABLE 4.6 – Répartition du jeu de données par classe

Classe	**Entraînement**	**Validation**	**Test**
fissure	2122	454	456
peinture	1051	225	226
humidite	2119	454	455
autre	1487	318	320
Total	**6779**	**1451**	**1457**

4.8 Prétraitement des données

Avant l'entraînement, plusieurs traitements ont été appliqués aux images afin de les rendre exploitables par les réseaux de neurones et d'améliorer la capacité de généralisation du modèle.

Redimensionnement. Toutes les images ont été ramenées à une résolution uniforme de 224×224 pixels en trois canaux de couleur (RVB), correspondant à la taille d'entrée standard des architectures pré-entraînées utilisées.

Normalisation spécifique. Chaque architecture pré-entraînée attend une mise à l'échelle particulière des valeurs de pixels. Cette normalisation a été intégrée directement à l'intérieur du modèle, via la fonction de prétraitement propre à chaque architecture. Ce choix garantit la cohérence entre l'entraînement et l'inférence sur mobile : l'application transmet de simples pixels bruts, et le modèle applique lui-même la transformation adéquate.

Augmentation des données. Afin de limiter le sur-apprentissage et d'enrichir la diversité visuelle de l'ensemble d'entraînement, une augmentation a été appliquée à la volée : retournement horizontal, légère rotation, zoom et variation de contraste. Ces transformations simulent des conditions de prise de vue variées, plus représentatives de l'usage réel de l'application.

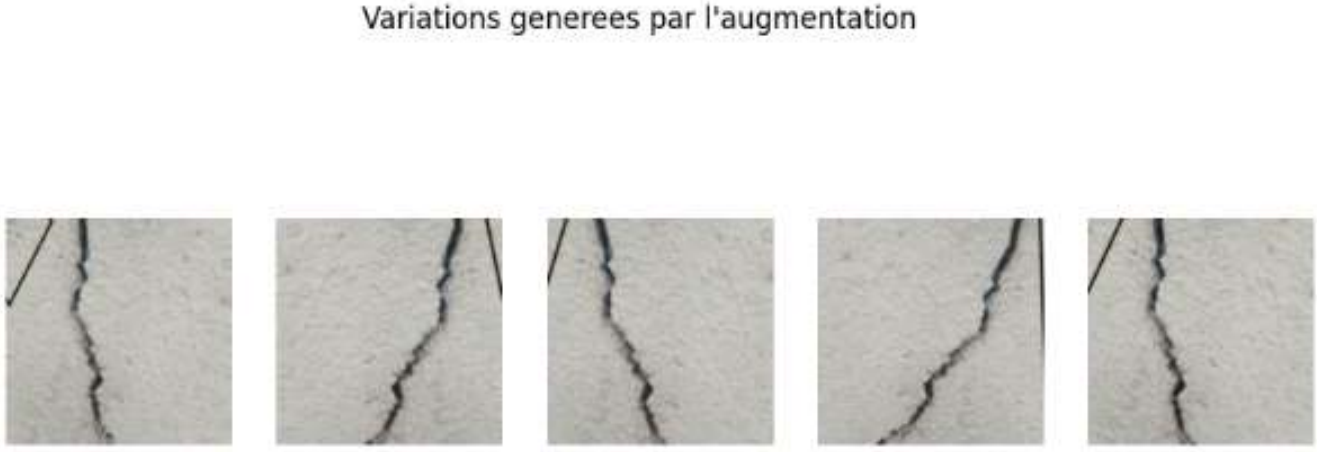


FIGURE 4.8 – Variations generees par l'augmentation

4.9 Architectures comparées

Quatre modèles ont été comparés afin d'identifier le meilleur compromis entre précision et contraintes d'embarquement. Le premier sert de référence (baseline), les trois suivants reposent sur l'apprentissage par transfert.

Réseau de neurones convolutif de référence (baseline CNN). Il s'agit d'un réseau convolutif simple, entraîné entièrement à partir de zéro. Il enchaîne trois blocs de convolution (32, 64 puis 128 filtres de taille 3×3 avec activation ReLU), chacun suivi d'une couche de sous-échantillonnage (*max pooling*), puis d'une couche d'agrégation globale, d'un *dropout* de 0,3 et d'une couche de classification finale. N'étant pas pré-entraîné, il fournit un point de comparaison permettant de mesurer l'apport réel de l'apprentissage par transfert.

MobileNetV2. Architecture spécifiquement conçue pour les environnements à ressources limitées tels que les appareils mobiles. Elle compte environ 3,5 millions de paramètres et repose sur des convolutions séparables en profondeur (*depthwise separable convolutions*) ainsi que sur des blocs résiduels inversés, qui réduisent fortement le nombre de paramètres et le coût de calcul tout en conservant une bonne capacité de représentation.

EfficientNetB0. Architecture comptant environ 5,3 millions de paramètres,

qui optimise conjointement la profondeur, la largeur et la résolution du réseau selon une stratégie de mise à l'échelle composée (*compound scaling*). Elle offre généralement un excellent rapport entre précision et nombre de paramètres.

ResNet50. Réseau résiduel profond de cinquante couches comptant environ 25,6 millions de paramètres. Il introduit des connexions de saut (*skip connections*) qui facilitent l'entraînement des réseaux profonds en limitant la disparition du gradient. Très performant en précision, il est en revanche nettement plus volumineux, ce qui le rend moins adapté à un déploiement embarqué.

Pour chaque architecture pré-entraînée, la base convolutive a d'abord été gelée et seule la couche de classification a été entraînée, afin de comparer équitablement les quatre candidats dans les mêmes conditions.

4.10 Métriques d'évaluation

L'évaluation des modèles s'appuie sur plusieurs métriques complémentaires, adaptées à un problème de classification multiclasse. Ces métriques se calculent à partir des quatre quantités fondamentales d'une classification : les vrais positifs (VP), les vrais négatifs (VN), les faux positifs (FP) et les faux négatifs (FN).

Exactitude (*accuracy*). Proportion globale de prédictions correctes sur l'ensemble de test. Elle donne une vue d'ensemble mais peut masquer des déséquilibres entre classes.

$$\text{Exactitude} = \frac{VP + VN}{VP + VN + FP + FN} \tag{4.1}$$

Précision (*precision*). Pour une classe donnée, proportion des prédictions de cette classe qui sont effectivement correctes. Elle mesure la fiabilité d'une prédiction positive.

$$\text{Précision} = \frac{VP}{VP + FP} \tag{4.2}$$

Rappel (*recall*). Pour une classe donnée, proportion des exemples réels de cette classe qui ont été correctement identifiés. Il mesure la capacité du modèle à

ne pas manquer les cas pertinents.

$$\text{Rappel} = \frac{VP}{VP + FN} \tag{4.3}$$

Score F1. Moyenne harmonique de la précision et du rappel. Il fournit un indicateur équilibré, particulièrement utile lorsque les classes ne sont pas parfaitement équilibrées, comme c'est le cas pour la classe *peinture*, moins représentée.

$$\text{F1} = 2 \times \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}} \tag{4.4}$$

Matrice de confusion. Représentation tabulaire croisant les classes réelles et les classes prédites. Chaque ligne correspond à une classe réelle et chaque colonne à une classe prédite ; la diagonale contient les prédictions correctes. Elle permet d'identifier précisément les confusions entre catégories.

4.11 Résultats et comparaison des architectures

Le tableau 4.7 présente les résultats de la comparaison des quatre architectures, en intégrant non seulement l'exactitude mais aussi deux critères essentiels pour un déploiement embarqué : la taille du modèle et le temps d'inférence par image.

TABLE 4.7 – Comparaison des architectures sur l'ensemble de test

Modèle	**Exactitude**	**Taille**	**Temps / image**
Baseline CNN	53,2 %	–	–
MobileNetV2	95,1 %	2,5 Mo	71,5 ms
EfficientNetB0	95,1 %	4,6 Mo	74,3 ms
ResNet50	96,8 %	23,9 Mo	78,0 ms

Plusieurs observations se dégagent de ces résultats. D'abord, l'apport de l'apprentissage par transfert est manifeste : le réseau de référence entraîné à partir

de zéro plafonne à 53,2 % d'exactitude, tandis que les trois architectures pré-entraînées dépassent toutes 95 %. Ensuite, ResNet50 obtient la meilleure exactitude (96,8 %), mais au prix d'une taille de près de 24 Mo, soit environ dix fois celle de MobileNetV2. Enfin, MobileNetV2 et EfficientNetB0 atteignent une exactitude identique (95,1 %), mais MobileNetV2 se distingue par un poids inférieur et un temps d'inférence légèrement plus court.

4.12 Choix du modèle et affinage

Le choix du modèle ne pouvait reposer sur la seule exactitude, car le modèle est destiné à fonctionner **directement sur l'appareil mobile de l'utilisateur**, sans appel à un serveur distant. Dans ce contexte, la taille du modèle et la rapidité d'exécution sont des critères déterminants, notamment pour les appareils d'entrée de gamme largement répandus au Maroc.

À précision égale avec EfficientNetB0, et pour un gain de seulement 1,7 point d'exactitude offert par ResNet50 mais au prix d'un poids dix fois supérieur, **MobileNetV2 constitue le meilleur compromis** entre performance et contraintes d'embarquement. C'est donc cette architecture qui a été retenue.

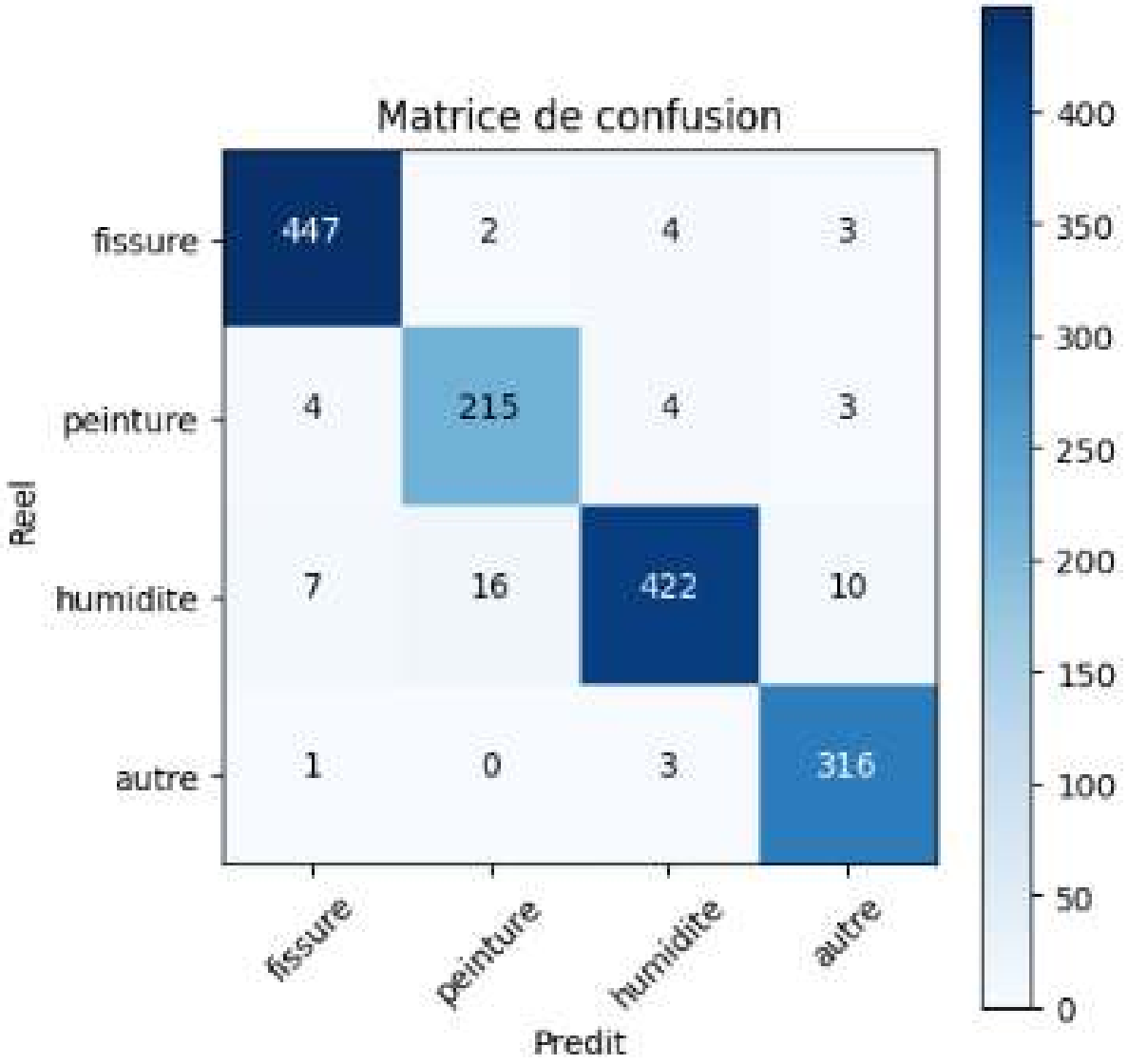
Le modèle retenu a ensuite fait l'objet d'un **affinage** : les dernières couches de la base convolutive ont été dégelées et réentraînées avec un taux d'apprentissage très faible, afin d'adapter plus finement les caractéristiques apprises sur ImageNet à notre domaine spécifique. Cet affinage a permis de porter l'exactitude de MobileNetV2 de 95,1 % à **96,1 %** sur l'ensemble de test.

Le tableau 4.8 détaille les performances du modèle final par classe.

TABLE 4.8 – Performances par classe du modèle final (MobileNetV2 affiné)

Classe	**Précision**	**Rappel**	**Score F1**	**Support**
fissure	0,974	0,980	0,977	456
peinture	0,923	0,951	0,937	226
humidite	0,975	0,927	0,950	455
autre	0,952	0,988	0,969	320
Exactitude globale	**0,961** (sur 1457 images)			

Les scores F1 sont élevés et homogènes sur l'ensemble des classes. La classe *peinture*, bien que moins représentée dans le jeu de données, conserve un score F1 satisfaisant de 0,937. La classe *autre* obtient un rappel élevé de 0,988, ce qui confirme l'efficacité du mécanisme de rejet des images hors-domaine.



	precision	recall	f1-score	support
fissure	0.974	0.980	0.977	456
peinture	0.923	0.951	0.937	226
humidite	0.975	0.927	0.950	455
autre	0.952	0.988	0.969	320
accuracy			0.961	1457
macro avg	0.956	0.962	0.958	1457
weighted avg	0.961	0.961	0.961	1457

FIGURE 4.9 – Matrice de confusion

4.12.1 Choix du seuil de confiance

Pour fiabiliser le pré-remplissage automatique de la demande, un seuil de confiance a été introduit : en deçà de ce seuil, le modèle s'abstient de proposer une catégorie et invite l'utilisateur à la saisir manuellement. Le tableau 4.9 présente l'effet du seuil sur la couverture (proportion de demandes automatiquement renseignées) et la précision des prédictions retenues.

TABLE 4.9 – Effet du seuil de confiance sur la couverture et la précision

Seuil	**Couverture**	**Précision**
0,60	98,6 %	98,3 %
0,70	97,3 %	98,4 %
0,80	95,7 %	98,7 %
0,90	93,0 %	99,0 %

Un seuil de **0,70** a été retenu pour le déploiement : il permet de renseigner automatiquement 97,3 % des demandes avec une précision de 98,4 %, soit le meilleur compromis entre l'assistance apportée à l'utilisateur et la fiabilité des suggestions.

4.13 Exportation pour le déploiement embarqué

Le modèle final a été converti au format **TensorFlow Lite**, format optimisé pour l'inférence sur appareils mobiles. Une optimisation par quantification des poids a été appliquée, réduisant la taille du fichier tout en préservant l'exactitude. Le modèle exporté occupe environ **2,4 Mo**, ce qui le rend parfaitement compatible avec une intégration directe dans l'application Android, y compris sur des appareils aux ressources modestes.

L'inférence s'exécute entièrement sur l'appareil, sans connexion réseau : la photographie fournie par le citoyen est redimensionnée, transmise au modèle, et

la catégorie prédite — si la confiance dépasse le seuil retenu — pré-remplit automatiquement le formulaire de demande de service.

4.14 Conclusion

Ce chapitre a présenté la démarche complète de conception du modèle d'intelligence artificielle intégré à Maalem. La comparaison de quatre architectures a confirmé l'intérêt de l'apprentissage par transfert et conduit au choix de MobileNetV2, qui offre le meilleur équilibre entre précision et légèreté pour un déploiement embarqué. Après affinage, le modèle atteint une exactitude de 96,1 % sur quatre classes, pour un poids inférieur à 2,5 Mo. L'ajout d'une classe de rejet et d'un seuil de confiance renforce la fiabilité du système en conditions réelles. Le modèle ainsi obtenu remplit pleinement son objectif : assister le citoyen dans l'expression de son besoin tout en préservant sa liberté de correction manuelle.

Chapitre 5

Réalisation

Introduction

Après avoir effectué l'analyse et la conception, nous nous intéressons dans ce chapitre à la réalisation (implémentation) de l'objectif de ce travail. Nous commençons par les différents outils et frameworks utilisés, puis nous illustrons le résultat obtenu à travers quelques captures d'écran des principales interfaces de l'application Maalem.

5.1 Outils utilisés

Cette section présente les principaux outils et frameworks mobilisés tout au long du développement de Maalem, en précisant le rôle concret de chacun dans notre projet.

5.1.1 Environnement de développement

Android Studio : nous avons développé l'intégralité de l'application mobile sous Android Studio, en tirant parti de son émulateur pour tester les écrans sur différentes tailles d'appareils et de ses outils de profilage pour optimiser les performances de l'app.



Visual Studio Code : il nous a servi d'éditeur principal pour le développement de l'interface d'administration web en React, notamment pour sa légèreté et son intégration native avec Git.



Google Colab : nous avons utilisé Colab pour entraîner notre modèle de classification d'images, en profitant de l'accès gratuit aux GPU pour accélérer l'entraînement sans contrainte matérielle locale.



5.1.2 Outils de conception

StarUML : il nous a permis de réaliser l'ensemble des diagrammes UML du projet (cas d'utilisation, séquence et classes) qui ont structuré notre phase de conception.



5.1.3 Développement mobile

Kotlin : c'est le langage avec lequel nous avons codé toute la logique de l'application mobile, en l'associant à l'architecture MVVM pour organiser proprement la couche présentation et la logique métier.



XML : nous l'avons utilisé pour définir les layouts des différents écrans de l'application (formulaires de demande, listes d'offres, profils, chat).



5.1.4 Développement web

React.js : nous avons développé le tableau de bord de l'administrateur avec React, en exploitant son approche par composants réutilisables pour construire les vues de supervision, de validation des artisans et de gestion des utilisateurs.



5.1.5 Backend et base de données

Firebase : il constitue le backend serverless de Maalem. Nous avons utilisé *Cloud Firestore* pour stocker et synchroniser en temps réel les demandes, offres et messages, *Authentication* pour l'identification des utilisateurs par OTP SMS, *Storage* pour héberger les photos des demandes de service et les portfolios, et *Cloud Messaging* pour notifier les utilisateurs.



5.1.6 Intelligence artificielle

TensorFlow / Keras : nous nous en sommes servies pour construire et entraîner notre modèle de classification d'images basé sur l'architecture EfficientNetB0, qui reconnaît la nature du problème domestique à partir d'une photo.



5.1.7 Gestion de versions

GitHub : nous avons utilisé GitHub pour versionner notre code, suivre l'évolution du développement et collaborer de manière organisée entre les membres de l'équipe.



5.2 Présentation des interfaces

Cette section présente les principales interfaces de l'application Maalem. Pour des raisons de clarté, nous regroupons d'abord les écrans communs liés à l'authentification, puis nous détaillons les interfaces propres à chacun des trois acteurs : le citoyen, l'artisan et l'administrateur.

5.2.1 Interfaces communes : authentification

Le module d'authentification constitue le point d'entrée commun à tous les utilisateurs de la plateforme. Il s'appuie sur Firebase Authentication avec la méthode email/mot de passe.

L'écran de connexion permet à l'utilisateur de saisir ses identifiants pour accéder à son espace. Selon le rôle associé à son compte dans Firestore, l'application le redirige automatiquement vers l'interface correspondante (citoyen ou artisan).

L'écran d'inscription permet la création d'un nouveau compte. L'utilisateur choisit son type de profil, renseigne ses informations personnelles et définit son mot de passe. Lors de l'inscription d'un artisan, des champs supplémentaires relatifs à la spécialité et à la zone d'intervention sont demandés, et le compte est placé en attente de validation par l'administrateur.

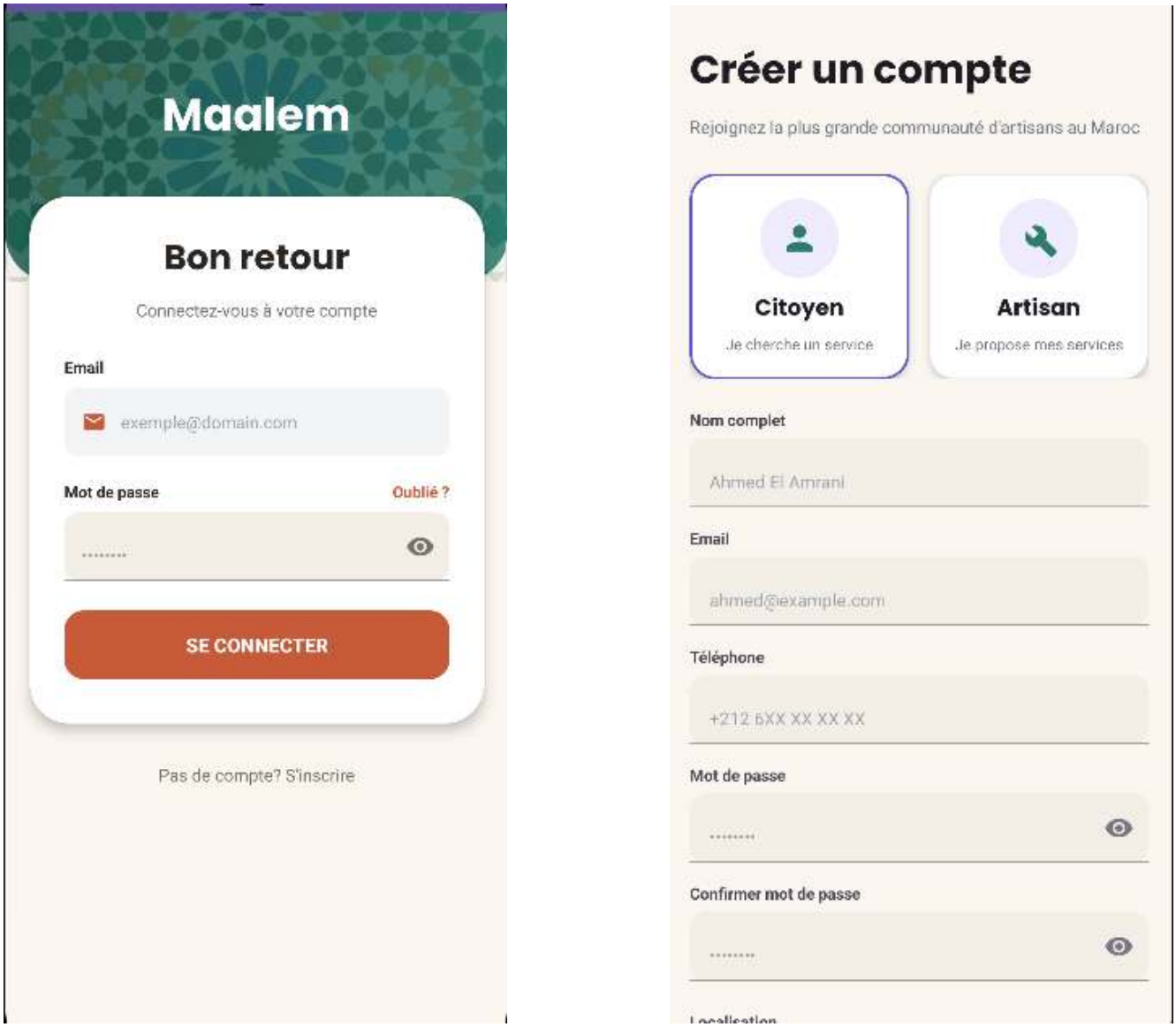


FIGURE 5.10 – Interfaces de connexion et d'inscription

5.2.2 Interfaces du citoyen

L'espace citoyen est conçu pour permettre une mise en relation simple et rapide avec les artisans.

L'écran d'accueil présente les services disponibles et donne accès à la création d'une nouvelle demande. La création d'une demande s'appuie sur une fonctionnalité d'intelligence artificielle : le citoyen peut téléverser une photo du défaut constaté (fissure, peinture écaillée, humidité), et le modèle embarqué classifie automatiquement le problème afin de pré-remplir le formulaire et d'orienter la

demande vers la spécialité adéquate.

L'écran de suivi des demandes permet au citoyen de consulter l'état de ses requêtes et de comparer les offres reçues de la part des artisans. Un écran de profil lui permet enfin de gérer ses informations personnelles et de consulter son historique.

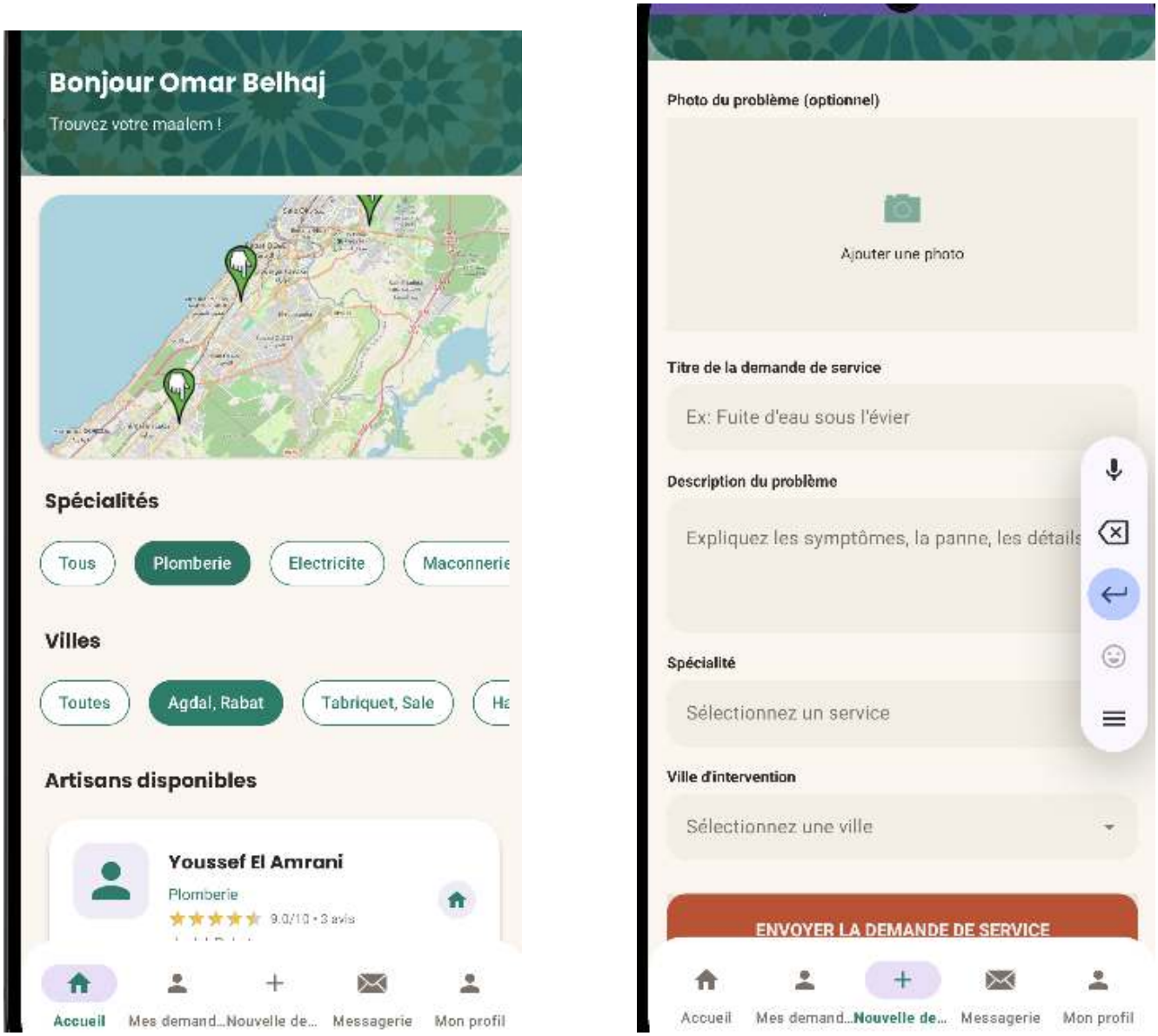


FIGURE 5.11 – Écran d'accueil et création d'une demande (espace citoyen)

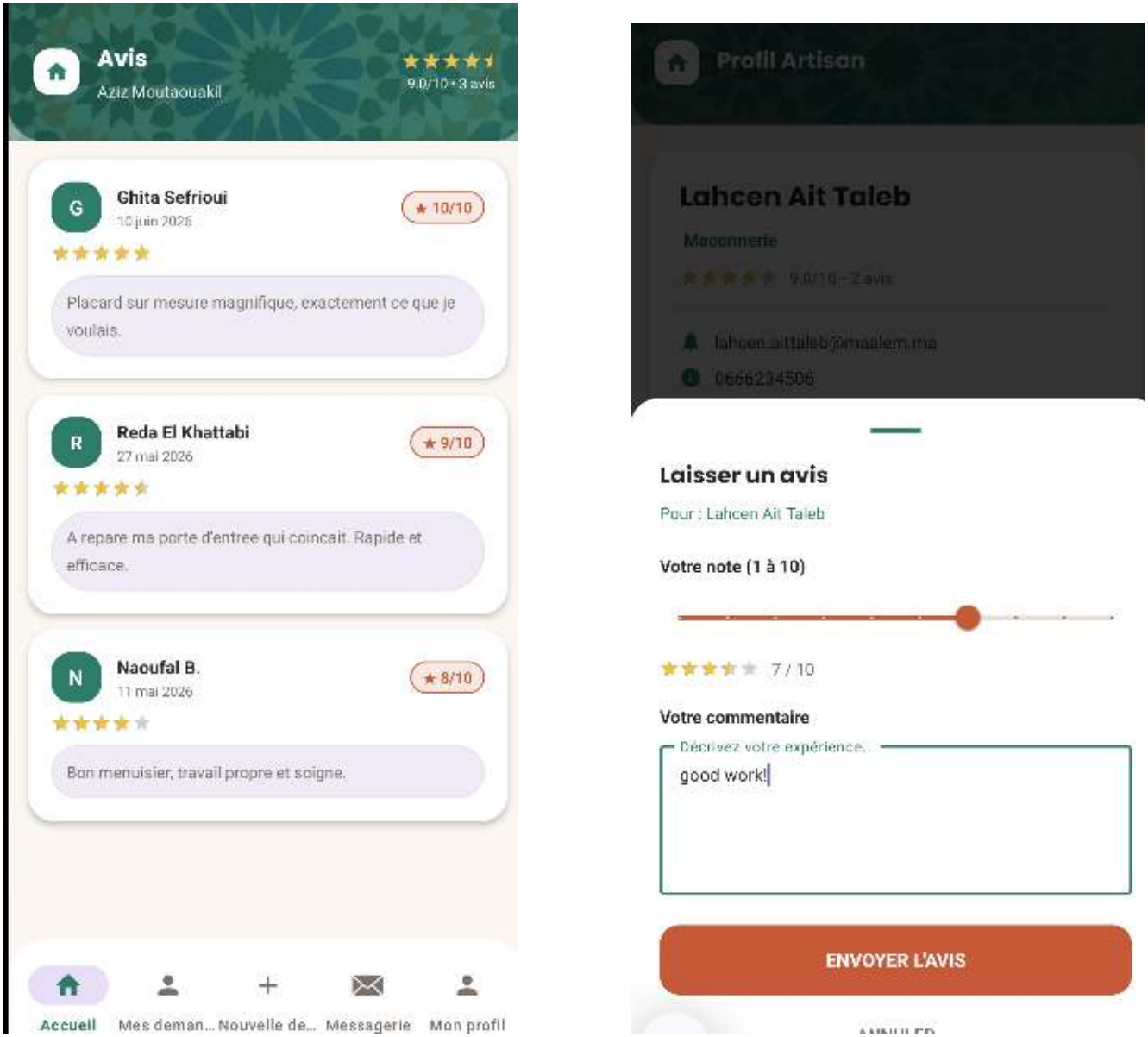


FIGURE 5.12 – Voir les avis sur un artisan et laisser un avis (espace citoyen)

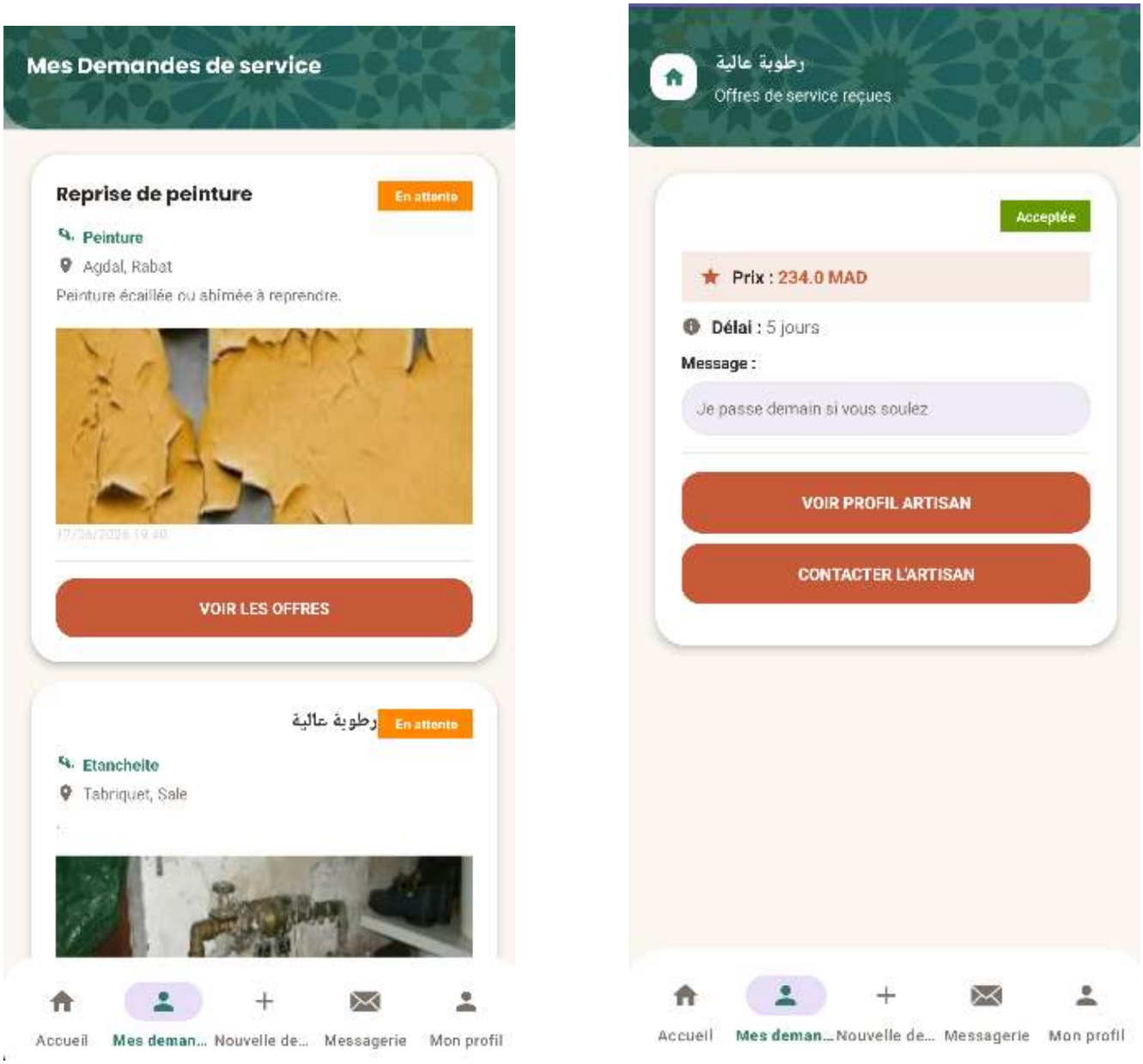


FIGURE 5.13 – Suivi des offres de service(espace citoyen)

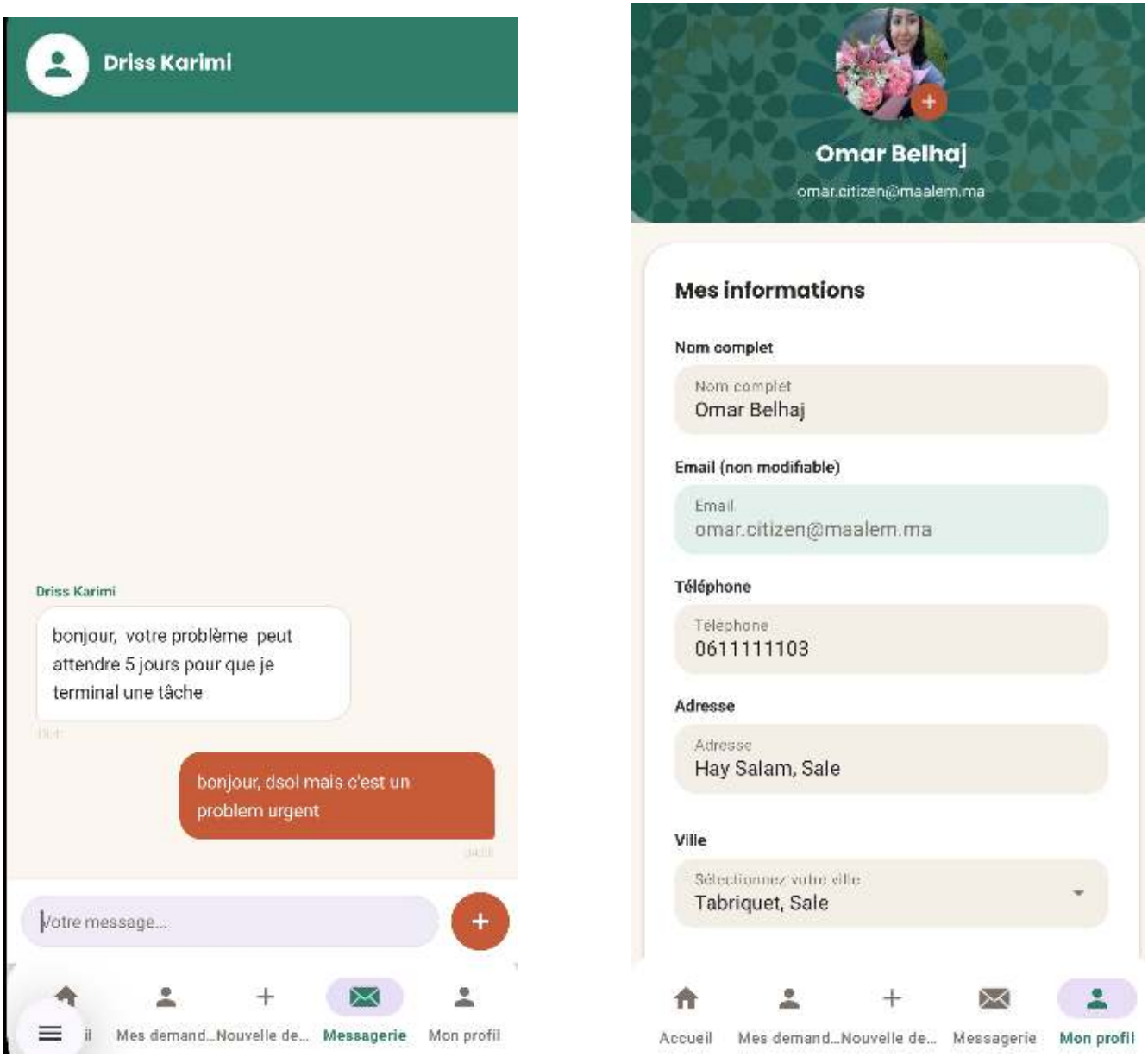


FIGURE 5.14 – Echanger via messagerie et profil (espace citoyen)

5.2.3 Interfaces de l'artisan

L'espace artisan regroupe les fonctionnalités permettant aux professionnels de gérer leur activité sur la plateforme.

Après validation de son compte par l'administrateur, l'artisan accède à un tableau de bord présentant les demandes correspondant à sa spécialité et à sa zone géographique. Il peut consulter le détail de chaque demande et y répondre en soumettant une offre comportant un descriptif de la prestation et une estimation

tarifaire.

L'écran de gestion des offres lui permet de suivre l'ensemble de ses propositions et leur statut. Son profil professionnel affiche ses informations, sa spécialité ainsi que sa note moyenne et les avis laissés par les citoyens.

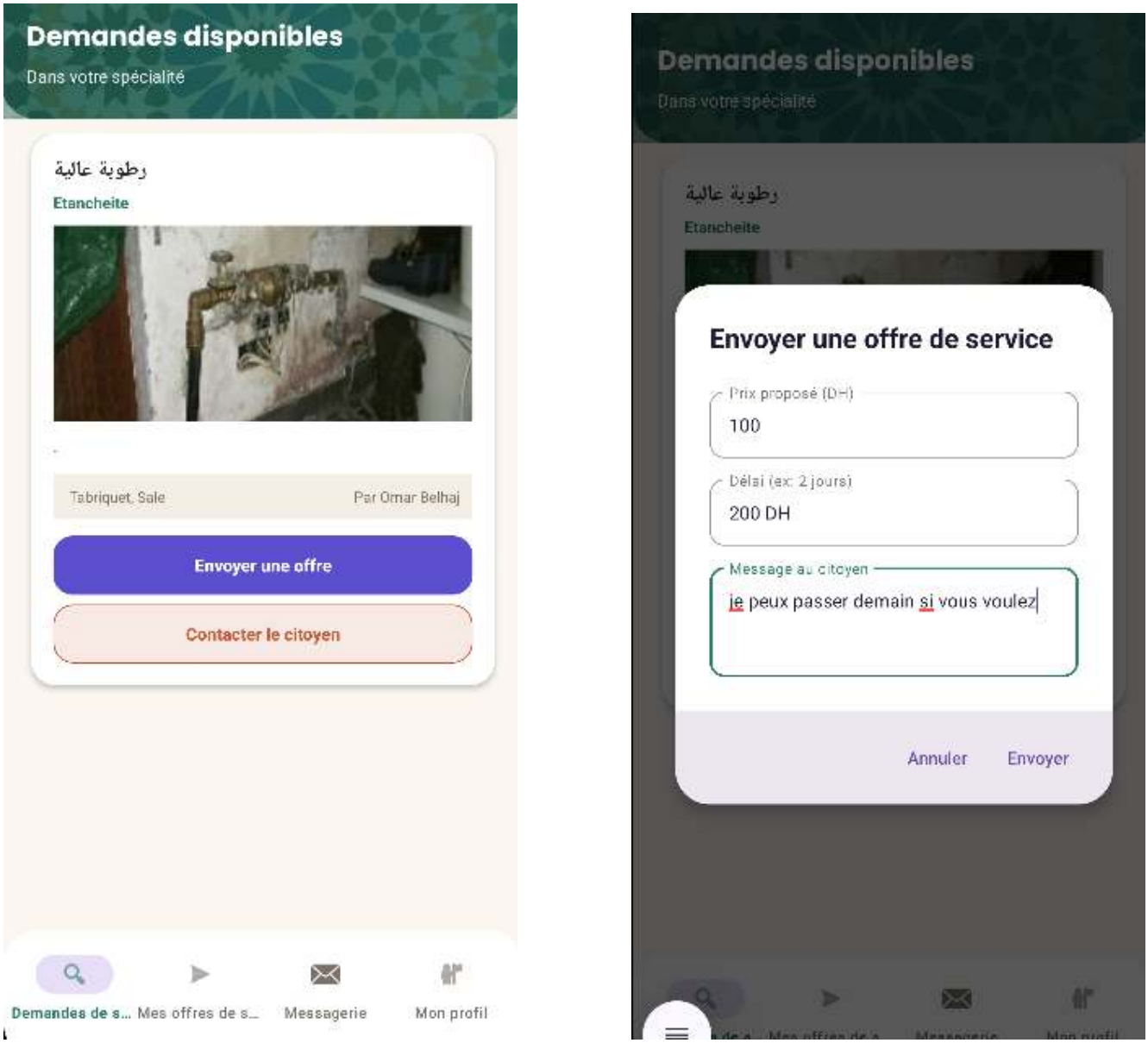


FIGURE 5.15 – Tableau de bord et soumission d'une offre (espace artisan)

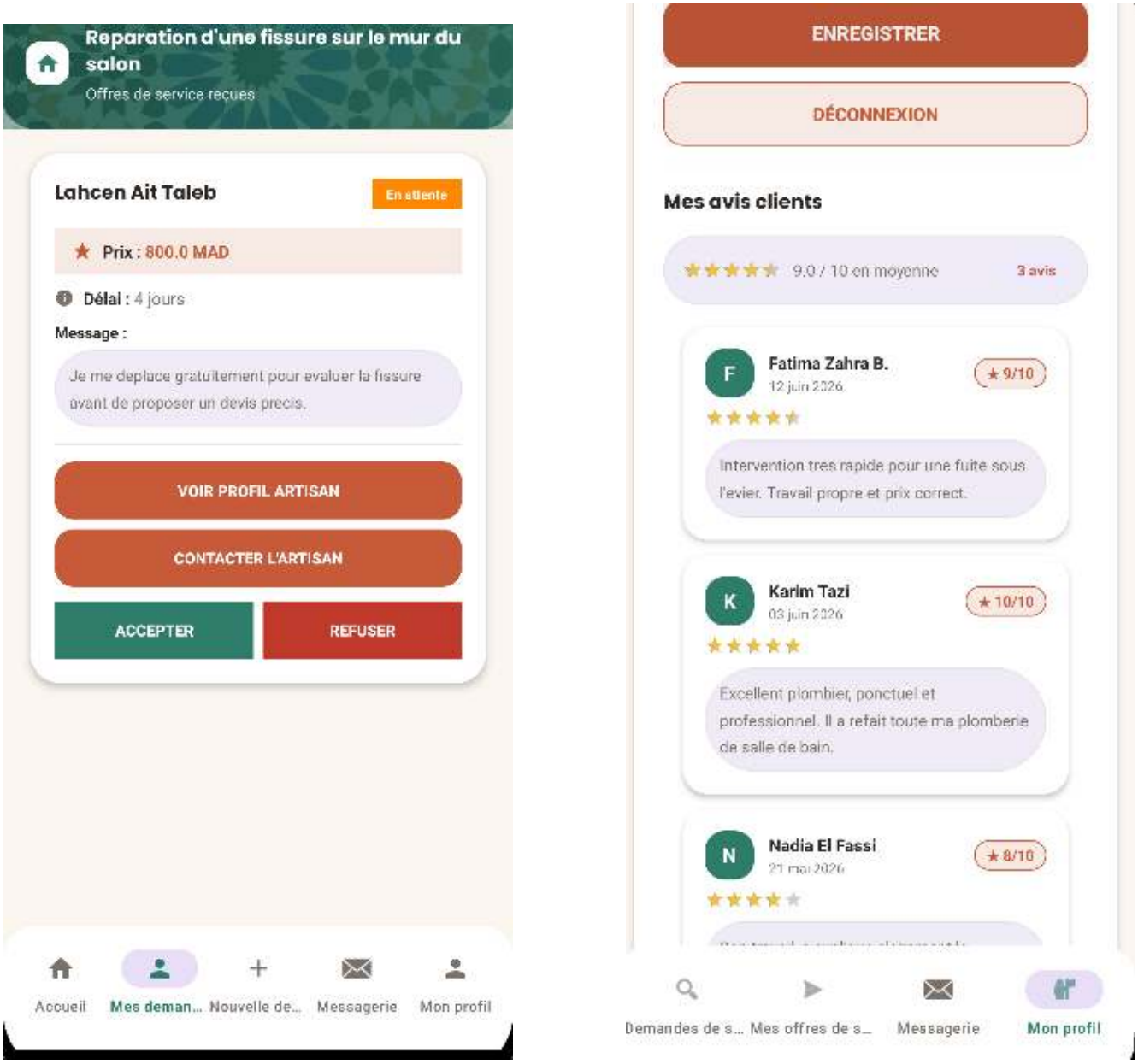


FIGURE 5.16 – Gestion des offres de service et profil professionnel (espace artisan)

5.2.4 Interfaces de l'administrateur

L'administrateur dispose d'un tableau de bord. Cette interface lui permet de superviser l'ensemble de la plateforme. La fonctionnalité centrale concerne la validation des comptes artisans : l'administrateur examine les demandes d'inscription en attente, vérifie les informations fournies et approuve ou rejette chaque compte.

Le tableau de bord offre également une vue d'ensemble des utilisateurs, des demandes et des offres, ainsi que des outils de gestion permettant de maintenir la

qualité et la fiabilité du service.

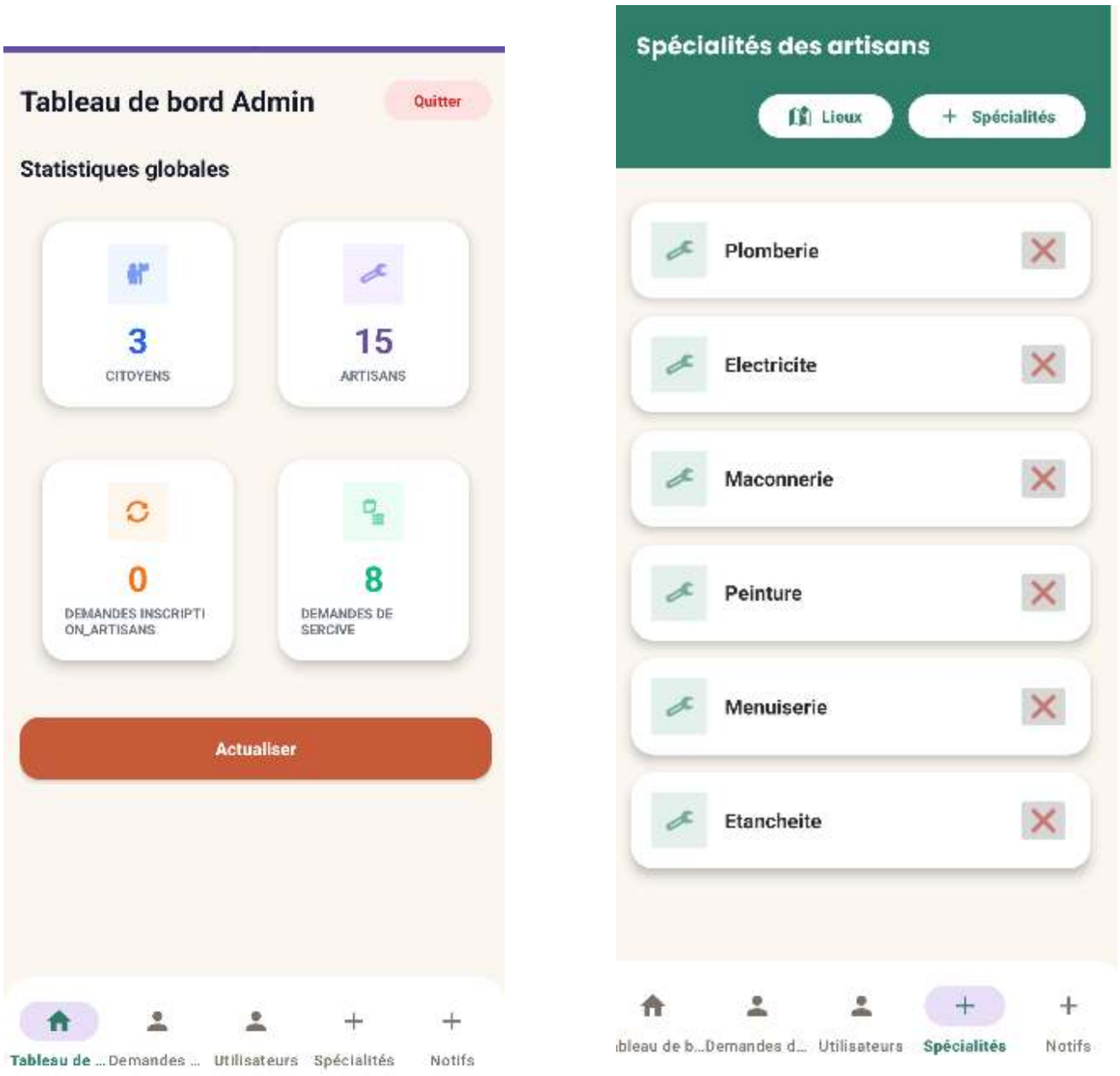


FIGURE 5.17 – Tableau de bord et gestion des spécialités et lieux (espace administrateur)

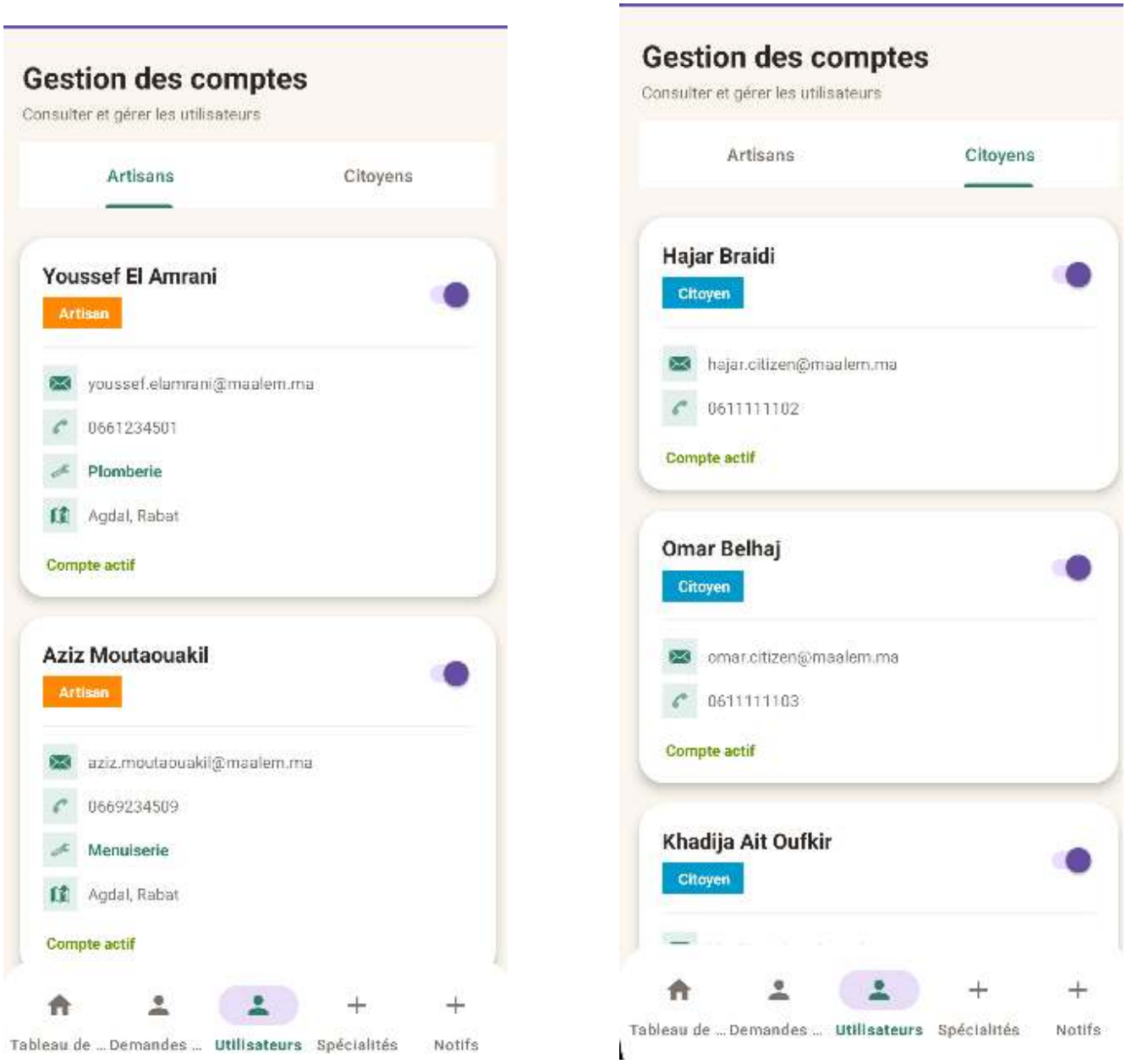


FIGURE 5.18 – Activer/ Désactiver les comptes(espace administrateur)

5.3 Conclusion

Dans ce chapitre, nous avons présenté la phase de réalisation de l'application Maalem. Nous avons illustré le résultat obtenu à travers les principales interfaces de l'application, démontrant ainsi la concrétisation des choix de conception établis dans les chapitres précédents.

Conclusion générale

Ce projet de fin d'année a abouti au développement de Maalem, une application mobile visant à structurer et moderniser la mise en relation entre les citoyens et les artisans qualifiés au Maroc. Face à un secteur encore largement informel, reposant sur le bouche-à-oreille et dépourvu de garanties de qualité, de disponibilité et de transparence tarifaire, notre solution apporte une réponse concrète en offrant une plateforme fiable, intuitive et évolutive.

Tout au long de ce travail, nous avons suivi une démarche rigoureuse, depuis l'étude des besoins et l'analyse des solutions existantes jusqu'à la réalisation, en passant par une conception soignée appuyée sur la modélisation UML et le patron architectural MVVM inspiré de la Clean Architecture. L'application repose sur trois profils complémentaires — le citoyen, l'artisan et l'administrateur — et intègre des fonctionnalités clés telles que la publication de demandes de service, un système d'offres de service structuré, une messagerie en temps réel, un système d'évaluation et une interface d'administration web.

L'apport le plus distinctif de Maalem réside dans l'intégration d'un modèle d'intelligence artificielle embarqué, fondé sur l'architecture MobileNetV2 et déployé via TensorFlow Lite. Grâce à une démarche expérimentale comparant plusieurs architectures, nous avons obtenu un modèle léger et performant capable de classifier automatiquement les types de pannes domestiques à partir d'une simple photo, simplifiant ainsi considérablement l'expression des besoins des citoyens.

En perspective, plusieurs axes d'évolution peuvent être envisagés pour enrichir la solution : l'extension du modèle d'IA à de nouvelles catégories de pannes,

l'intégration d'un système de paiement en ligne sécurisé et la géolocalisation avancée des artisans en temps réel. Ces évolutions visent à faire de Maalem un outil durable et performant, contribuant pleinement à la digitalisation du secteur artisanal marocain.

Bibliographie

[1] Bricole, *Application de mise en relation avec des artisans au Maroc*, Google Play, 2024. `https://bricoleapp.com` (consulté le 20/02/2026).

[2] Saweblia, *Service de dépannage et réparation à domicile*, 2024. `https://saweblia.ma` (consulté le 20/02/2026).

[3] TaskRabbit, *Same Day Handyman, Moving & Delivery Services*, IKEA, 2024. `https://taskrabbit.com` (consulté le 20/02/2026).

[4] Google, *Android Developers Documentation*, Google LLC, 2024. `https://developer.android.com/docs` .

[5] JetBrains, *Kotlin Language Documentation*, JetBrains s.r.o., 2024. `https://kotlinlang.org/docs/home.html` .

[6] Google, *Firebase Documentation*, Google LLC, 2024. `https://firebase.google.com/docs` .

[7] React Team, *React – A JavaScript library for building user interfaces*, Meta Open Source, 2024. `https://react.dev/` .

[8] Google, *TensorFlow Lite Guide – On-device machine learning*, Google LLC, 2024. `https://www.tensorflow.org/lite/guide` .

[9] Object Management Group, *Unified Modeling Language (UML) Specification, Version 2.5.1*, OMG, 2017. `https://www.omg.org/spec/UML/`.

Annexe

.1 Modèle conceptuel des données

Le modèle conceptuel ci-dessous a une vocation **purement informative**. Il représente les entités métier de l'application, leurs attributs et les relations qui les lient, afin d'offrir une vision claire et structurée du domaine. Il ne s'agit pas d'un schéma d'implémentation : comme expliqué dans le corps du rapport, Maalem ne repose pas sur une base relationnelle mais sur une base de données NoSQL orientée documents.

L'entité centrale **User** regroupe les informations communes à tous les utilisateurs (nom, prénom, email, téléphone, rôle, date de création, photo et statut). À partir de cette entité de base, deux profils spécialisés se distinguent : le **Citoyen**, caractérisé par sa ville et son adresse, et l'**Artisan**, qui ajoute des attributs métier tels que la spécialité, la disponibilité, la note moyenne et la CIN.

Le **Citoyen** est l'acteur qui exprime un besoin : il publie une ou plusieurs **Demandes de service**. La **Demande de service** contient les informations essentielles (titre, description, catégorie, date de création, statut et ville).

En réponse à une demande de service, un **Artisan** envoie une **Offre de service**, qui regroupe le prix proposé, le délai d'exécution, le statut, un message descriptif et la date d'envoi. Le citoyen peut ainsi comparer les différentes offres reçues et accepter ou refuser chacune d'entre elles.

La communication entre les acteurs est assurée par l'entité **Messagerie**, qui enregistre le contenu des messages échangés ainsi que leur date d'envoi, permettant un dialogue en temps réel entre le citoyen et l'artisan autour d'une demande.

À l'issue d'une prestation, le citoyen évalue le travail de l'artisan via l'entité **Avis**, qui contient une note, un commentaire et une date, afin de mettre à jour la note moyenne de l'artisan et de construire sa réputation sur la plateforme.

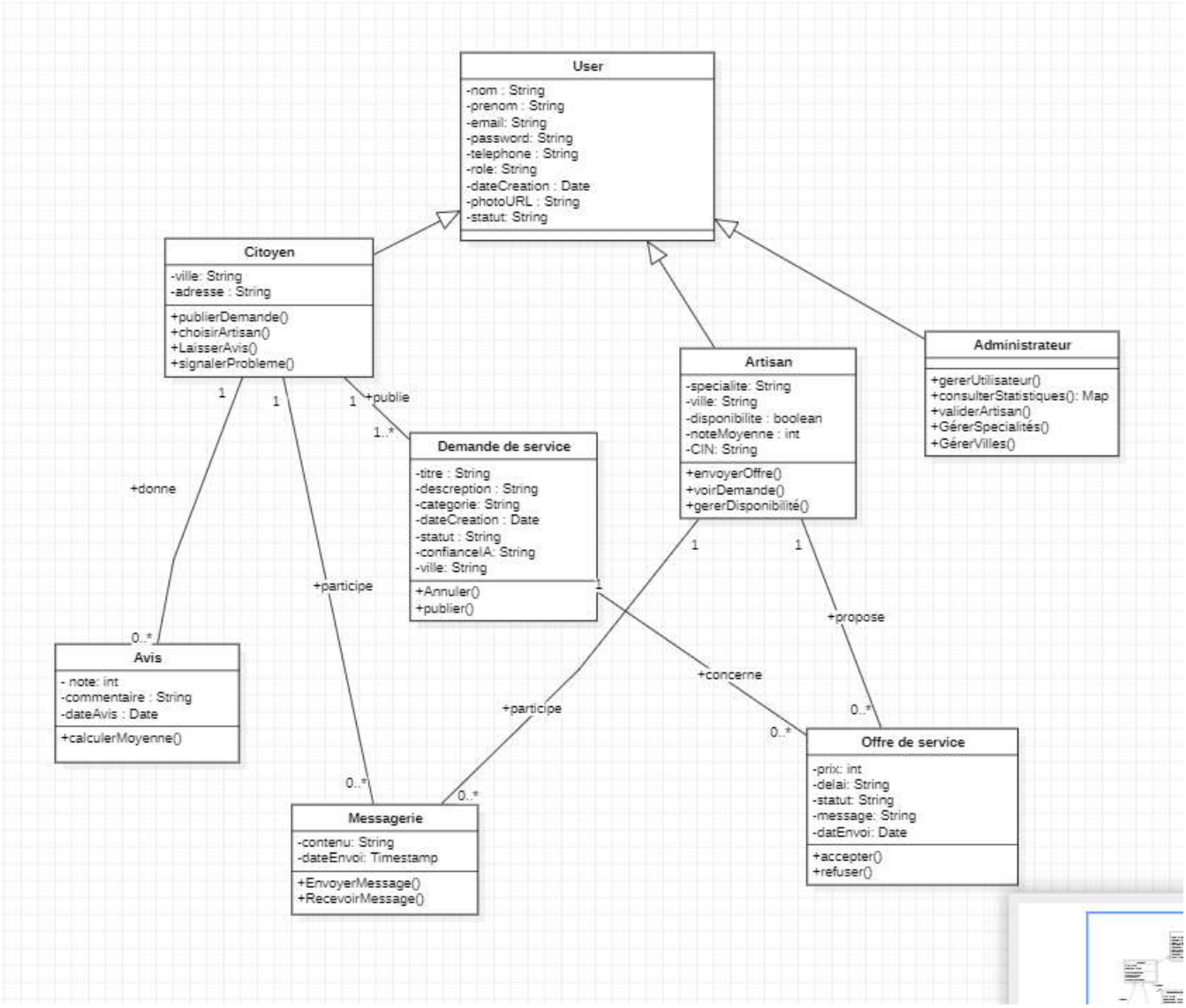


FIGURE 19 – Modèle conceptuel des données de l'application Maalem